

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

Aula 1 e 2

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2º Semestre

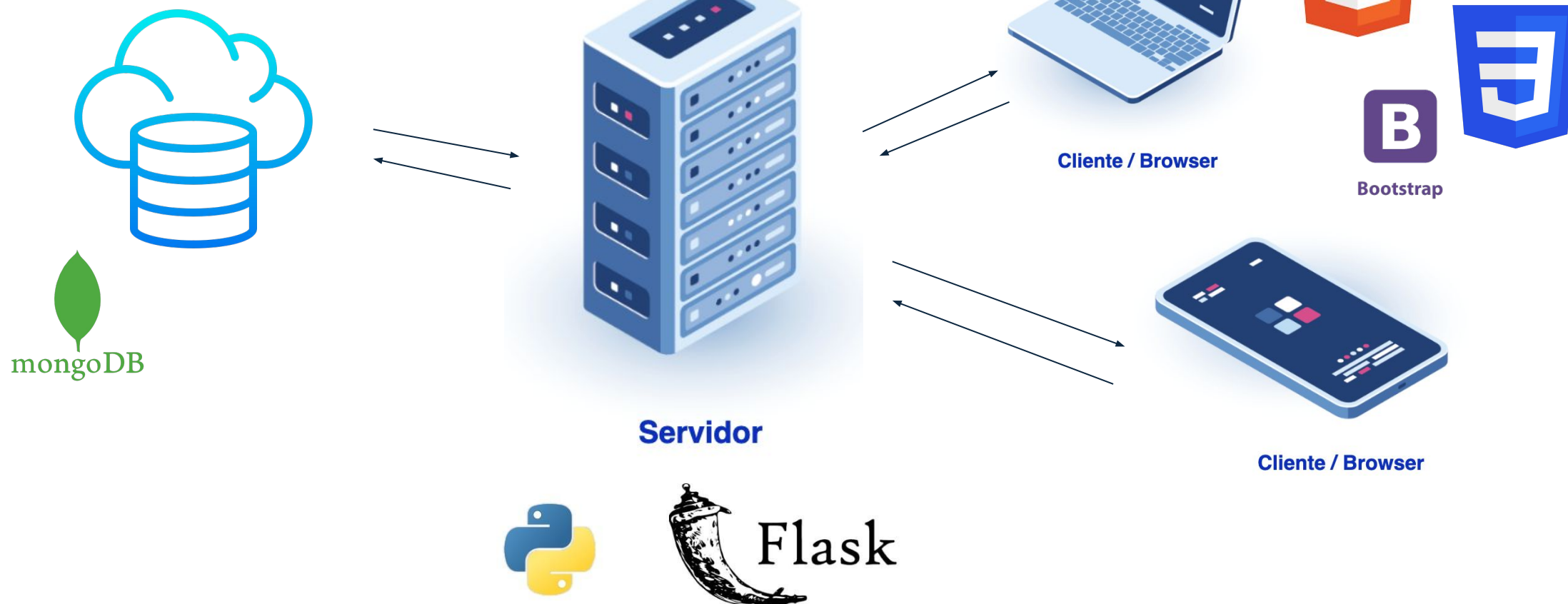
João Matos

- Horário de atendimento: Quarta-feira e Sexta-feira das 10h00 às 11h30
- joao_miguel_goncalves_matos@iscte-iul.pt
- Nota: O estudante deverá avisar o docente até à véspera caso pretenda estar presente no horário de dúvidas.

Objetivos / Programa

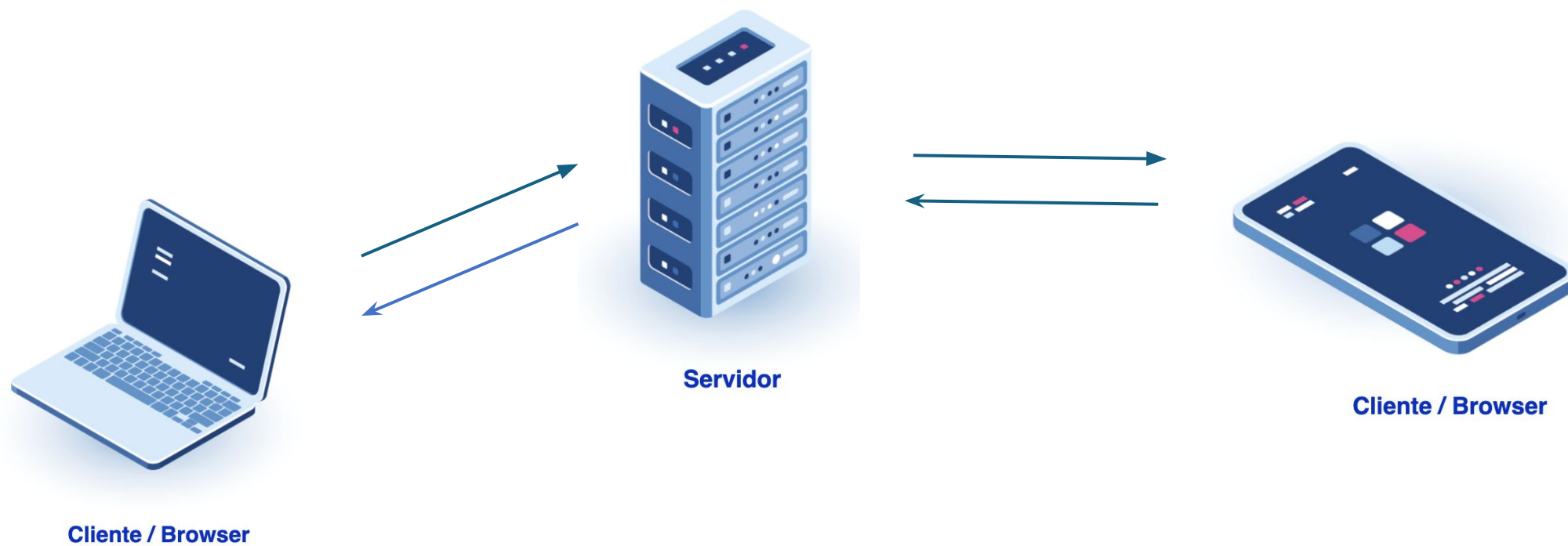
- Compreender tecnologias web, linguagens de programação e arquiteturas mais usuais
- Identificar e analisar diferentes tipos de arquitetura na nuvem e principais características
- Desenvolver Aplicações FrontEnd e Backend
 - Arquitetura WWW
 - Modelo Cliente-Servidor
 - Protocolo HTTP
 - Métodos GET, POST, PUT, etc.
 - HTML / CSS
 - Introdução Javascript
 - Node JS / Módulos NPM
 - FrontEnd framework - React JS
 - Base de dados relacionais e não-relacionais
 - MongoDB
 - RESTful API
 - Backend framework – Flask / Django (Python)

Programa



- 2 Projetos
 - o Cada projeto vale 50% da avaliação final
 - o Grupos de 2 - 4 alunos
 - o Primeiro Projeto: Desenvolvimento de uma Aplicação FrontEnd (React)
 - o Segundo Projeto: Desenvolvimento de uma Aplicação Backend (Python)
 - o Apresentação dos projetos com discussão individual
 - Data 1º Exame

Arquitetura Cliente - Servidor



WWW Arquitetura

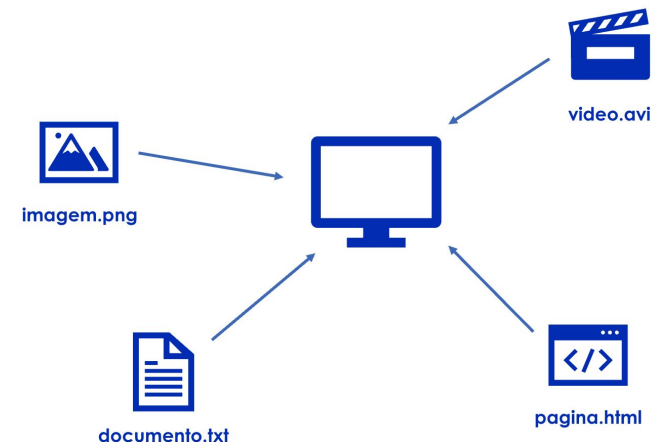
Construído em torno de 3 conceitos:

- **URL** (Uniform Resource Locator) - endereços que permitem identificar cada elemento (documentos, imagens, videos, etc.) de forma única;
- **HTTP** (Hypertext Transfer Protocol) - protocolo principal usado para transferir dados. Especifica como é feito o envio de informação entre cliente-servidor;
- **HTML** (Hypertext Markup Language): define a estrutura e os conteúdos de uma página web (texto, imagens, links, etc.)

Conteúdos

Onde estão os conteúdos?

- Podem estar em máquinas diferentes;
- Em pastas diferentes dentro da mesma máquina;
- Como podemos localizar os conteúdos/recursos?



A localização dos recursos (imagens, documentos, vídeos, áudios, etc.) é feita através do **URL**:

`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

protocolo

máquina/host

caminho/path

documento

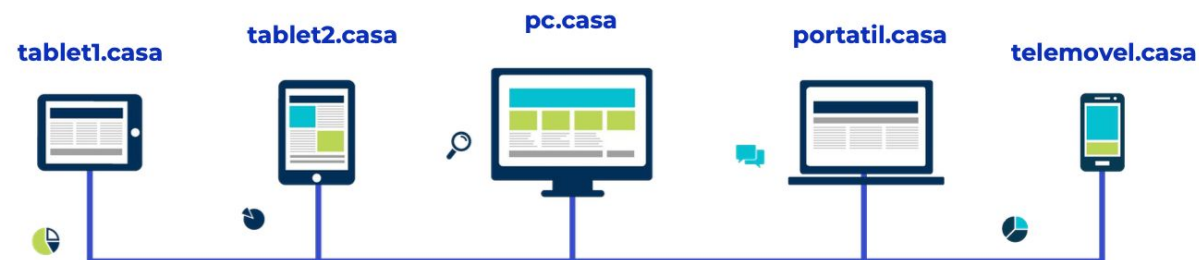
Endereço IP - DNS

IP (Internet Protocol) - número de identificação de cada máquina na internet



DNS foi criado para facilitar o acesso aos diferentes servidores/máquinas da rede, sem termos de saber o IP.

DNS - catálogo de endereços IP que localiza os servidores com base no nome de domínio;



`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

máquina/host

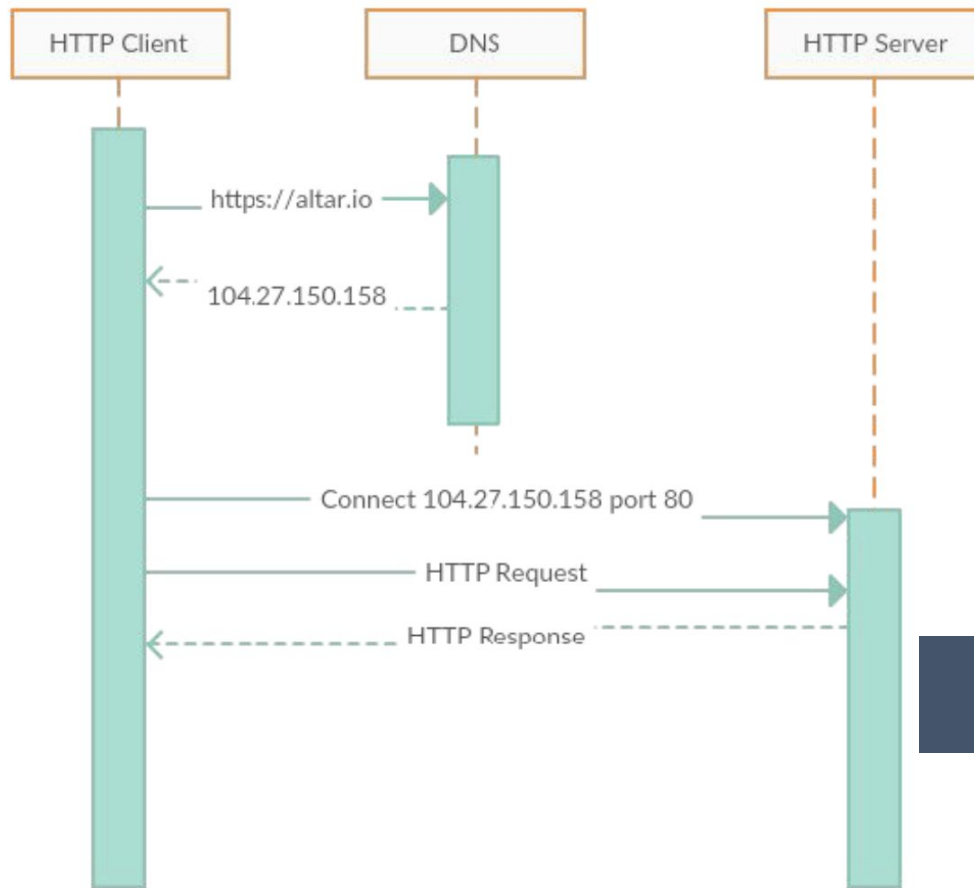
Para encontrar a "máquina" onde está o ficheiro (imagem1.jpeg) primeiro temos de saber o IP da máquina;

O DNS (Domain Name System) é a solução - O DNS traduz nomes de domínio em endereços IP;

Exemplo:

- google.pt: 142.250.201.67
- maquina_dominio.pt: [193.136.189.102](#)

Encontrar o conteúdo



- O navegador da web envia uma solicitação DNS para um servidor DNS.
- O servidor DNS consulta a sua base de dados ou outros servidores DNS para encontrar o endereço IP associado ao nome de domínio fornecido.
- Uma vez que o endereço IP é localizado, o servidor DNS retorna essa informação para o navegador.
- O navegador pode então estabelecer uma conexão com o servidor web utilizando o endereço IP retornado e solicitar os recursos desejados.

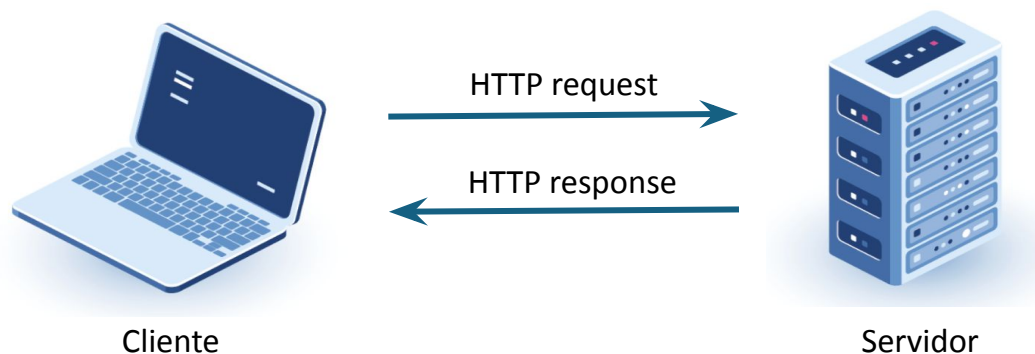
`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

`http://193.136.189.102/imagens/2024/imagem1.jpeg`

HTTP

"Hypertext Transfer Protocol"

- Implementado para permitir a comunicação *web browser / web server*
- Transmissão de documentos *hypermedia*
- Modelo cliente-servidor: cliente faz um *request* e espera pela resposta do servidor
- "*stateless protocol*" - não mantém/guarda o estado entre duas conexões
- Define como clientes Web requisitam páginas aos servidores e como estes transferem a informação

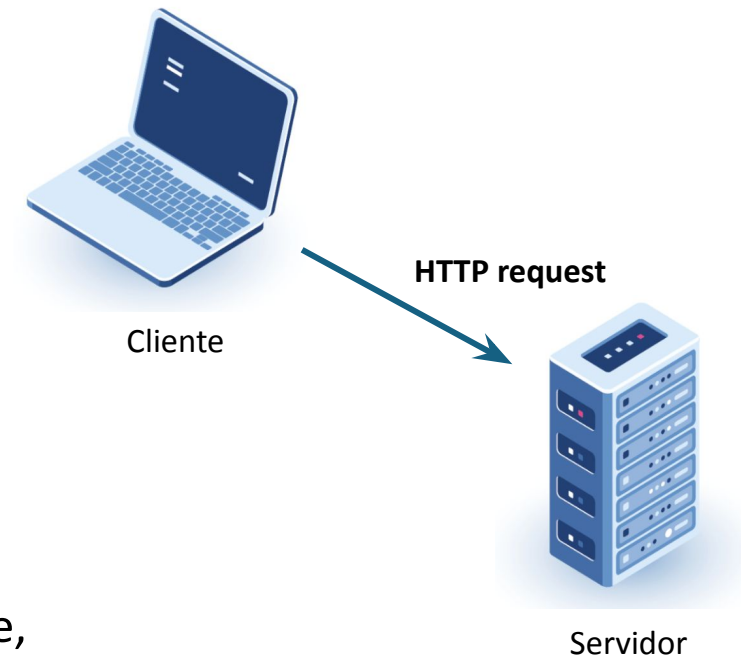


HTTP - request

HTTP Request – mensagem que um cliente envia a um servidor para requisitar recursos: páginas web, imagens, documentos, etc.

HTTP Request é composto por:

- Tipo de pedido: tipo de pedido efetuado ao servidor (GET, POST, etc.)
- URL: Endereço
- Cabeçalhos: informações adicionais a enviar ao servidor (Cache, Language, Content-type, etc.)
- Corpo: Dados enviados ao servidor para tratamento



HTTP - tipos de pedido

GET	Obter/pedir um recurso/dados ao servidor
POST	Criar um recurso/dados no servidor
PUT	Update um recurso existente no servidor
DELETE	Delete dados no servidor
PATCH	Update um recurso existente - não cria esse recurso
CONNECT	Efetuar conexão entre servidor (TCP/IP Tunnel)
OPTIONS	Pedir informações sobre comunicações a um <i>endpoint</i>
TRACE	Usado para teste/diagnóstico durante desenvolvimento
HEAD	Obter mensagem de cabeçalho sem os recursos

HTTP - GET

GET

Criar um recurso/dados no servidor

Método GET é usado para solicitar dados a um recurso específico no servidor. Cliente usa este método quando quer receber dados, como páginas web, imagens, conteúdos, etc.

- Pode ser cacheado pelo browser;
- Aparece no histórico de navegação
- Têm limite de dimensão
- Usado apenas para obter dados, não modificar

HTTP - GET

GET

Obter/pedir um recurso/dados ao servidor

Parâmetros GET:

- Parâmetros são enviados no endereço URL para especificar com maior detalhe os recursos a serem obtidos:

`http://host.com/books?ano=2000&author=JohnDoe`

Estrutura dos parâmetros GET
key/value pair

HTTP - POST

POST

Obter/pedir um recurso/dados ao servidor

Método POST é usado para criar ou alterar um recurso que está no servidor.

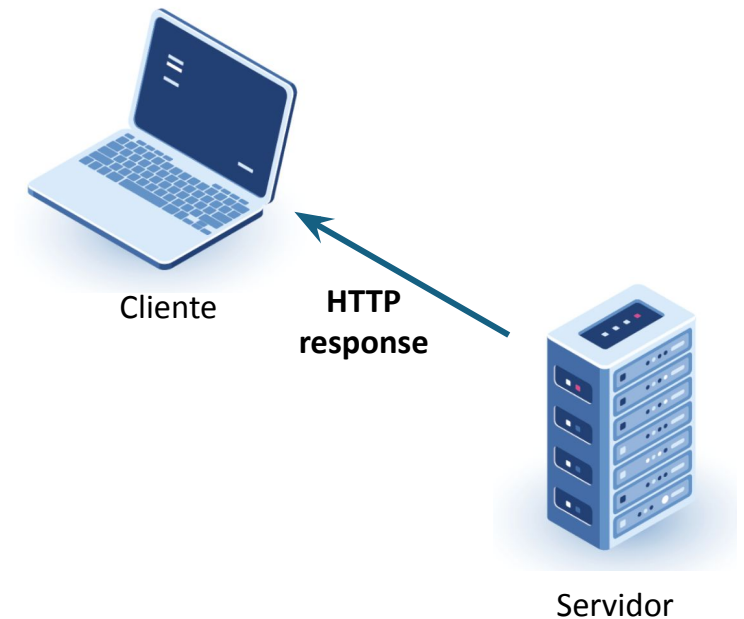
- Não podem ficar na cache do browser
- Não têm limite de dimensão (mas servidor pode limitar)

HTTP - Response

HTTP Response – mensagem que o servidor envia ao cliente depois de receber e processar o pedido

HTTP Response é composto por:

- Resultado/Status: código HTTP representativo de sucesso ou de erros na processamento do pedido
- Cabeçalhos: informações adicionais a enviar ao cliente para entender a resposta (codificação de conteúdo, etc.)
- Corpo: conteúdo da resposta HTTP



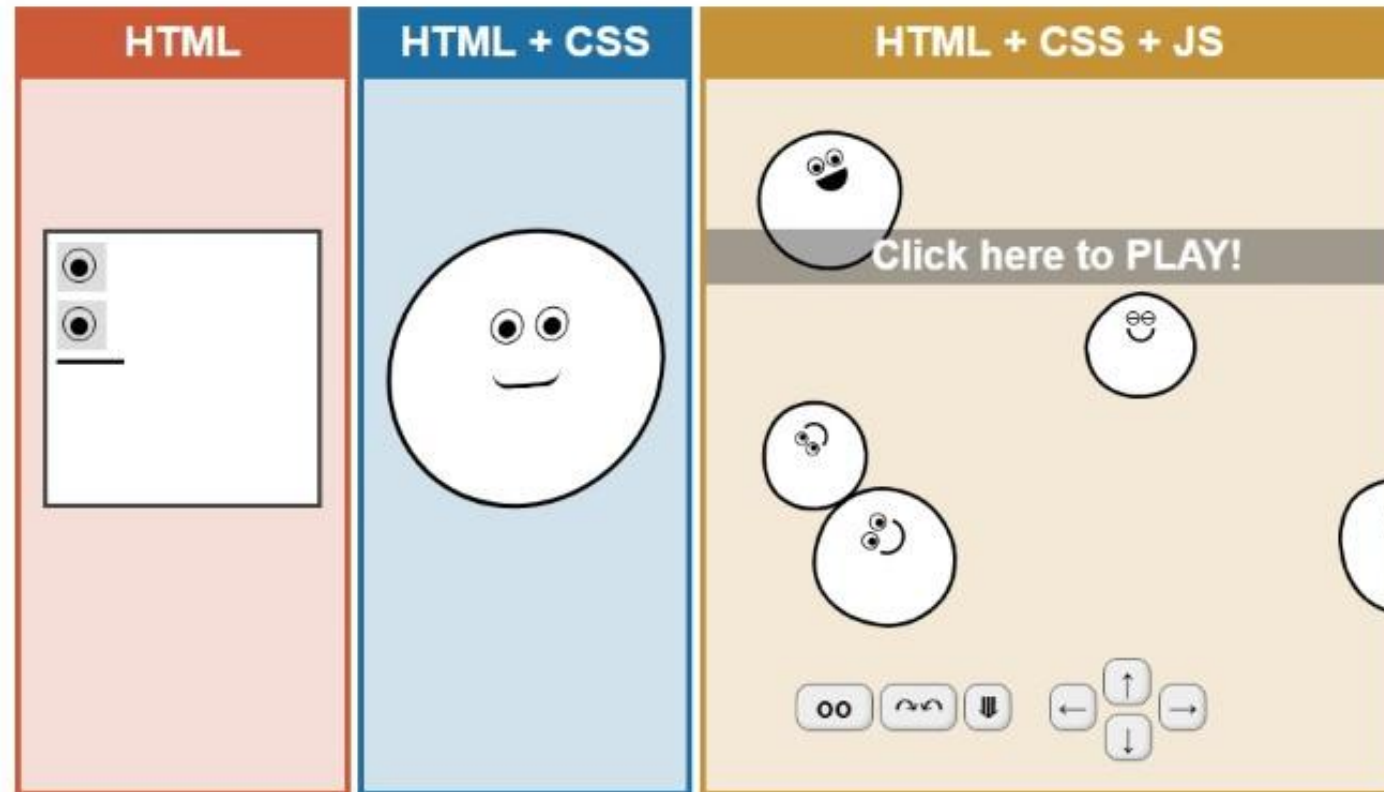
HTTP - Status message

Código de 3 dígitos que indica o resultado do pedido HTTP.

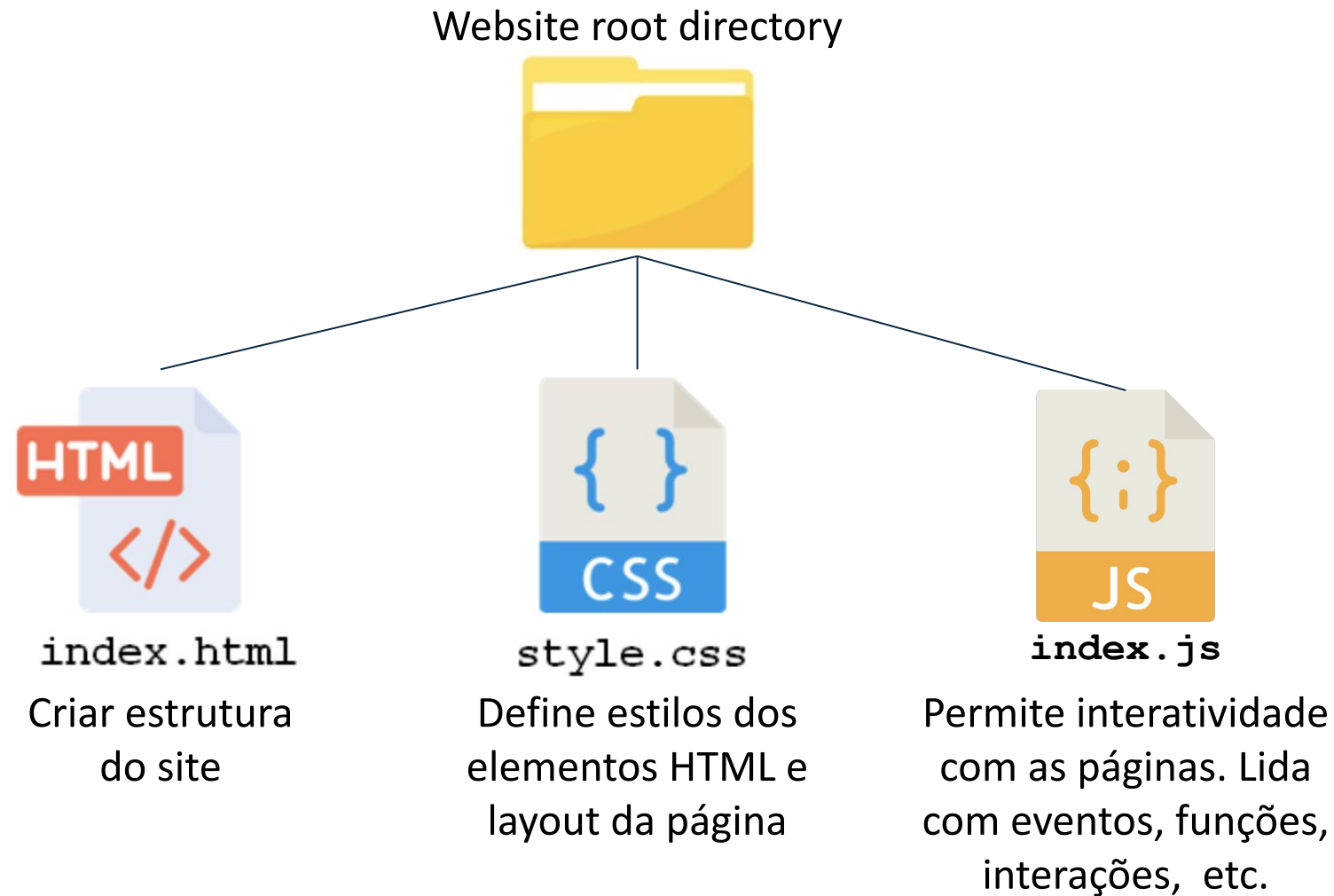
Exemplo de códigos comuns:

1XX Information	102 Processing
2XX Success	200 Success / OK
3XX Redirection	301 Permanent Redirect
4XX Client Error	404 Not Found 403 Forbidden
5XX Server Error	500 Internal Server Error 504 Gateway timeout

Como os sites são feitos



HTML + CSS + JS



"HyperText Markup Language"

- *Markup Language*, define a estrutura de como é apresentado o conteúdo/texto
- *HyperText*, define as ligações entre documentos, dentro do mesmo site ou para sites externos
- Descreve a estrutura de páginas Web
- Conjunto de diferentes elementos que dizem ao *browser* como apresentar ou dispor os conteúdos
- Não é considerado um linguagem de programação
- Os conteúdos ou a estrutura de cada página é definida por **tags**



Template HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Título da Página</title>
  </head>
  <body>
    <!-- Conteúdo da Página -->
  </body>
</html>
```

`<html>` tag principal que encapsula todo o documento

`<head>` tag que encapsula a informação "invisível" do documento (ficheiros CSS, código Javascript, etc.)

`<body>` tag de conteúdo que encapsula todo o conteúdo que pretendemos apresentar

Elementos HTML

Elemento HTML é definido por uma *tag* inicial, o conteúdo e uma *tag* final

`<h1>` O meu primeiro texto. `</h1>`

Tag de
abertura

Conteúdo

Tag de fecho

- Algumas Exceções: `<br>` ; `<hr>` ; `<img>`

Elementos HTML

Dentro de cada elemento HTML podemos ter outros elementos:

```
<p> Parágrafo sem ênfase no meio </p>
```

```
<p> Parágrafo com <strong>ênfase</strong> no meio </p>
```

Output:

Parágrafo sem ênfase no meio.

Parágrafo com **ênfase** no meio.

Atributos HTML

Atributos HTML adicionam informação extra a um elemento HTML

```
<p style="color: red"> Este texto é vermelho. </p>
```

Output:

Este texto é vermelho.

- Atributos têm de ser especificados na tag inicial
- Atributos são normalmente definidos com *"name/value" pairs*

Atributos HTML

Atributos HTML adicionam informação extra a um elemento HTML

```
<p style="color: red"> Este texto é vermelho. </p>
```

Atributos definem comportamentos, formatação, informação adicional, etc:

Output:

Este texto é vermelho.

Exemplos de atributos:

- *style* – permite aplicar estilos CSS ao elemento;
- *href* – define o URL de uma hiperligação;
- *target* – alterar o comportamento de um link (abrir num novo separador, etc.);
- *src* – define o URL de localização de uma image, video ou áudio;
- *class*
- *id*

Elementos HTML

Títulos HTML: `<h1>` até `<h6>`,
sendo `<h1>` a tag mais importante.

```
<h1> Título. </h1>  
<h2> Título. </h2>  
<h3> Título. </h3>
```

Parágrafos HTML:

```
<p> Parágrafo. </p>
```

Links/Hiperligações HTML: o link de destino é especificado
no atributo `href`

```
<a href="https://www.google.com"> Google </a>
```

Imagens HTML: `src` especifica o caminho até à imagem
; `alt` texto alternativo

```

```


Elementos DIV

- Permite agrupar múltiplos elementos HTML – *container*
- Usado para estruturar o layout das páginas
- Permite aplicar diferentes estilos a todos os elementos dentro do *container*

```
<div>
  <h1>Titulo.</h1>
  <p>Parágrafo</p>
  <a href="link.com">Link</a>
</div>
```

Titulo.

Parágrafo

[Link](#)

Hyperlinks

- Possibilita o encaminhamento para outra página
- Conteúdo que é "clicável" e encaminhado para outra localização

```
<a href="https://www.google.com">Google</a>
```

Atributo *href* – define o endereço do encaminhamento

Conteúdo - pode ser texto ou imagens

Pode ser encaminhado para um elemento dentro da própria página

```
<a href="about.html">About</a>  
<a href="#target1">Link 1</a>  
<a href="team.html">Team</a>  
<p>Lorem ipsum dolor sit amet, consectetur adipiscing elit.</p>  
<p id="target1">Lorem ipsum dolor sit amet, consectetur adipiscing  
elit.</p>
```

Listas

- *Unordered list*: `<ul>`
- *Ordered list*: `<ol>`

```
<ul>
  <li>Google</li>
  <li>Microsoft</li>
  <li>IBM</li>
</ul>
```

Output:

- Google
- Microsoft
- IBM

Atributo “type”, indica o tipo de ordenação:

type=”1” - lista por números

type=”A” - lista por letras maiúsculas

type=”a” - lista por letras minúsculas

type=”I” - lista por números romanos maiúsculos

type=”i” - lista por números romanos minúsculos

```
<ol>
  <li>Google</li>
  <li>Microsoft</li>
  <li>IBM</li>
</ol>
```

Output:

1. Google
2. Microsoft
3. IBM

Imagens

```

```



https://www.w3schools.com/html/html_images.asp

Tabelas

Representação de dados em forma de tabela composta por linhas e colunas

<tr>: define uma linha

<td>: define uma coluna

<th>: titulo de cada coluna

<thead> e <tbody> são opcionais

Nome	Sobrenome	Idade
João	Silva	28
Maria	Souza	28

```
<table>
  <thead>
    <tr>
      <td>Nome</td>
      <td>Sobrenome</td>
      <td>Idade</td>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>João</td>
      <td>Silva</td>
      <td>28</td>
    </tr>
    <tr>
      <td>Maria</td>
      <td>Souza</td>
      <td>28</td>
    </tr>
  </tbody>
</table>
```

Atributo Class

- Especifica o nome da *class* que pode ser usado por um ou vários elementos HTML
- Normalmente usado para especificar determinados estilos CSS
- Permite o acesso e/ou manipulação através de JavaScript

```
<div class="city">
  <h1>Lisboa</h1>
  <p>Capital de Portugal.</p>
</div>
```

```
.city {
  background-color: tomato;
  color: white;
  border: 2px solid black;
  margin: 20px;
  padding: 20px;
  width: 100px;
}
```



Atributo Class

- Elementos HTML podem ter mais de uma *class*
- Elemento terá os estilos definidos de todas as suas *classes*

```
<h2 class="city main">London</h2>  
<h2 class="city">Paris</h2>  
<h2 class="city">Tokyo</h2>
```

London

Paris

Tokyo

Atributo id

- Identifica um determinado elemento HTML ou uma secção da página
- Só pode existir um id único por página

```
<section id="ola">  
  <h2>Olá Mundo!</h2>  
</section>
```

```
<section id="outro_id">  
  <p>Esta é outra secção!</p>  
</section>
```


Preparação do *setup*



<https://code.visualstudio.com/>

PastaProjeto

|-- imagens

|-- css

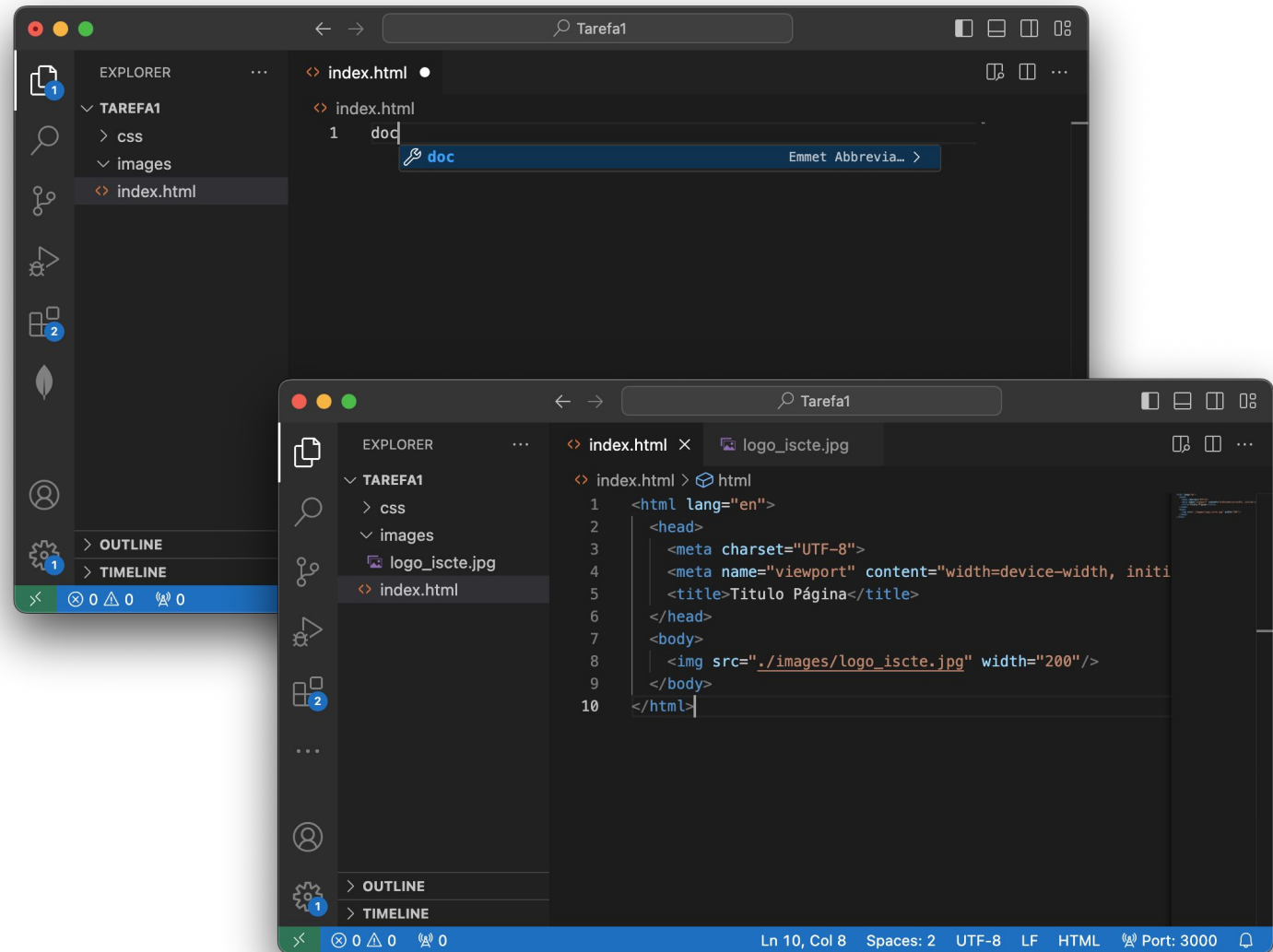
|-- index.css

|-- index.html

Caminho:

href="imagens/imagem.png"

href="./imagens/imagem.png"

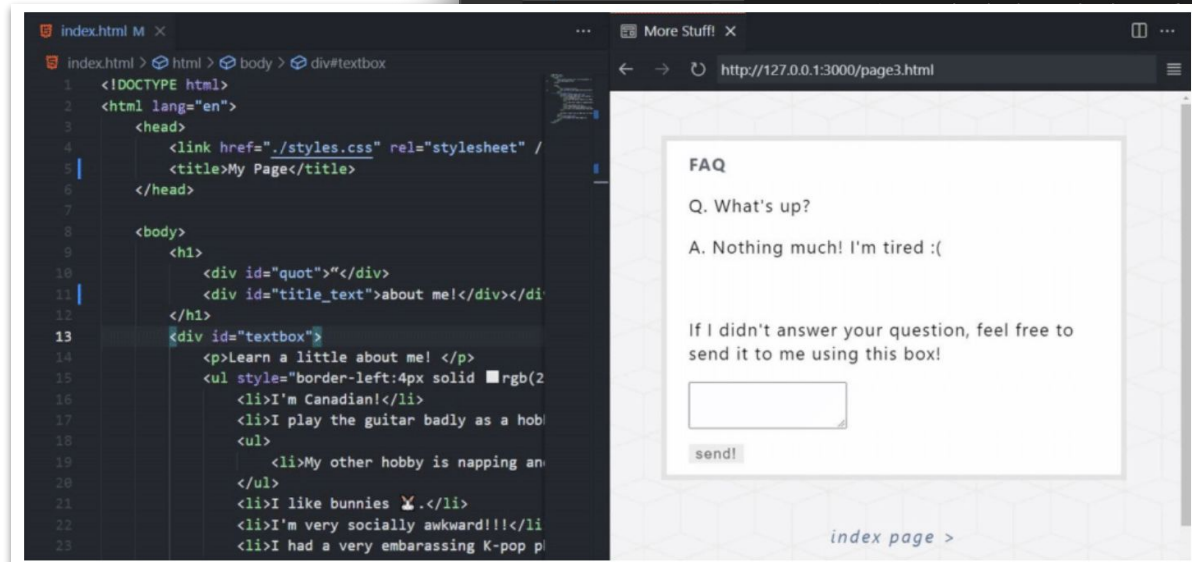
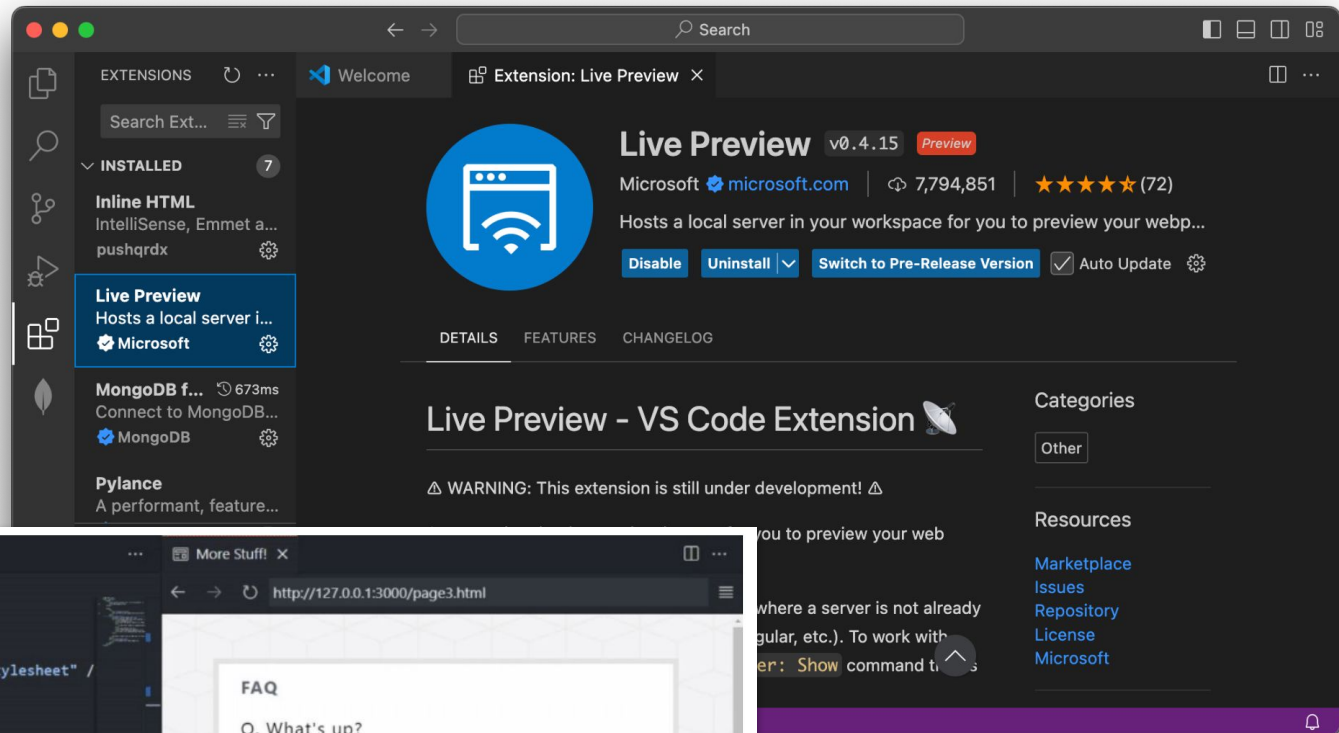


Preparação do setup



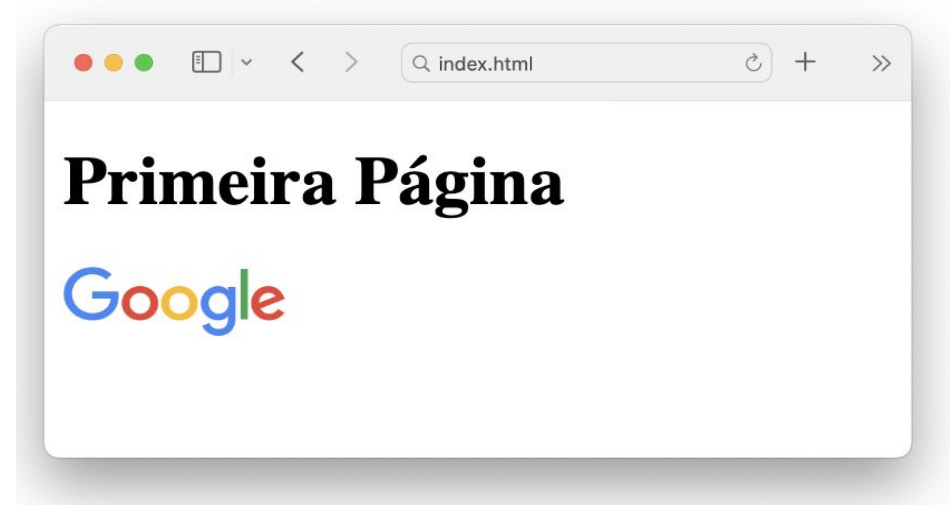
<https://code.visualstudio.com/>

Live Preview



Tarefa 1

- Criar uma página (index.html) com o seguinte conteúdo:
- Imagem clicável com link para o Google



Tarefa 2

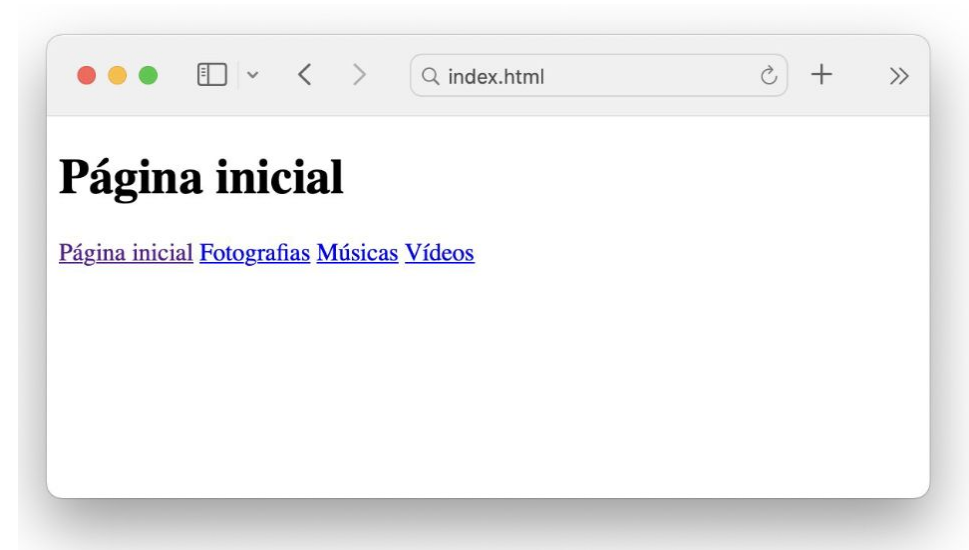
- Criar um conjunto de 4 ficheiros HTML (4 páginas)
- Criar link com navegação para as páginas

index.html - Página inicial

fotografias.html - Página Fotografias

musicas.html - Página Músicas

videos.html - Página Videos



Forms

- Elemento `<form>` é usado para aceitar dados inseridos do utilizador
- Informação é geralmente enviado para o servidor para ser processada (login, pesquisa, etc.)
- *Container* para diferentes tipos de input (text fields, checkboxes, radio buttons, submit buttons, etc)
- Não apresenta nenhuma referência visual quando criada

```
<form>
|   <!-- Campos formulário -->
</form>
```

Forms - atributos

- **action** - define a ação a ser executada assim que a form for submetida. Geralmente enviada para o servidor para os dados serem tratados.

```
<form action="/action_page.php">
  <label for="fname">Name:</label><br>
  <input type="text" id="fname" name="fname" value="John">
  <input type="submit" value="Submit">
</form>
```

- **method** - método HTTP usado para a submissão (get ou post)

```
<form action="/action_page.php" target="_blank" method="get"></form>
```

- **enctype** - especifica como os dados devem ser codificados antes de serem enviados ao servidor

```
<form action="/action_page.php" method="post" enctype="multipart/form-data"></form>
```

Forms - elementos

- Form possui campos que permite inserir dados
- Elemento `<input/>` tag mais comum

`<input type='text' />`

```
<input type="text" value="John">
```

John

`<input type='radio' />`

```
<input type="radio" id="html" name="fav_language" value="HTML">
<label for="html">HTML</label><br>
<input type="radio" id="css" name="fav_language" value="CSS">
<label for="css">CSS</label><br>
<input type="radio" id="javascript" name="fav_language" value="JavaScript">
<label for="javascript">JavaScript</label>
```

- ☒ HTML
- ☐ CSS
- ☐ JavaScript

Forms - elementos

`<input type='checkbox' />`

```
<input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
<label for="vehicle1"> I have a bike</label><br>
<input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
<label for="vehicle2"> I have a car</label><br>
<input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
<label for="vehicle3"> I have a boat</label>
```

- ☐ I have a bike
- ☐ I have a car
- ☐ I have a boat

`<input type='submit' />`

```
<input type="submit" value="Submit">
```

Submit

`<input type='button' />`

```
<input type="button" value="Button">
```

Button

Forms - elementos

```
<input type='email' />
```

```
<input type='search' />
```

```
<input type='url' />
```

Para dispositivos móveis ou tácteis o teclado apresenta caracteres úteis, como: “@”, ou “.com”.

Também haverá validação se não escrevermos um formato válido.

Number

```
<input type='number' />
```

Forms - elementos

Date, Time, Datetime-local

```
<input type="date" />
```

dd/mm/yyyy

September 2024

S	M	T	W	T	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5
6	7	8	9	10	11	12

Clear Today

```
<input type="time" />
```

--:--

14	32
15	33
16	34
17	35
18	36
19	37
20	38

```
<input type="datetime-local" />
```

dd/mm/yyyy, --:--

September 2024

S	M	T	W	T	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5
6	7	8	9	10	11	12

14	31
15	32
16	33
17	34
18	35
19	36
20	37

Clear Today

Forms - elementos

Range

```
<input type="range" />
```



Select

```
<select name="cars" id="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="opel">Opel</option>
  <option value="audi">Audi</option>
</select>
```

Volvo ▾



Forms - elementos

Atributos input

- **placeholder** - texto que aparece quando o campo está vazio;
- **maxlength** - quantidade máxima de caracteres;
- **required** - indica que o campo é necessário
- **disabled** - deixa o campo desabilitado
- **readonly** - impede editar o campo

- Cross-browser: garantir a compatibilidade com todos os dispositivos e com diferentes browsers. Muitas vezes temos de implementar código diferente para diferentes browsers.
- Responsive Web Design: O layout do website deve adaptar-se automaticamente aos diferentes tamanhos de ecrã dos dispositivos; Muitas vezes temos de mudar o layout da página ou remover texto/images;
- Performance: Páginas Web devem ser rápidas carregar. Escolher tamanho ideal de imagens; minimizar o código; etc.
- Accessibility: Website deve ser acessível a todos - adaptar conteúdo a pessoas com deficiências visuais, deficiências auditivas, incapacidades cognitivas ou incapacidades físicas.
- Internationalization
- Privacy & Security

- Usar elementos HTML de forma a fornecer significado estrutural e contextual para o conteúdo da página.
- Em vez de simplesmente usar elementos HTML com base na sua aparência visual, selecionar cuidadosamente os elementos que melhor descrevem a estrutura e conteúdo. - **HTML Semantic**
- Elementos que descrevem melhor o conteúdo: `<header>`, `<nav>`, `<article>`, `<section>`, `<footer>` - em vez de usar só `<div>` para todo o conteúdo
- SEO: mecanismos de busca utilizam a semântica HTML apropriados para entender melhor o contexto do conteúdo e melhor no posicionamento dos resultados de pesquisa
- Cada elemento tem um propósito semântico claro que ajuda na manutenção do website

Práticas Web - exemplo

<header>

<section>

<footer>

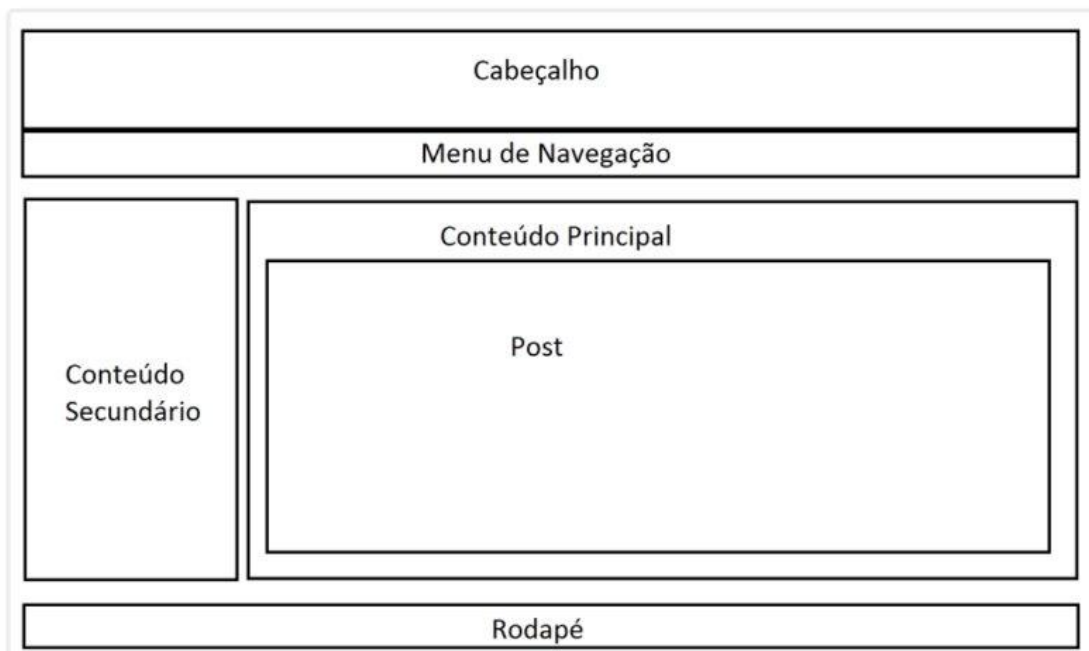


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplo de HTML Semântico</title>
</head>
<body>
  <header>
    <h1>Meu Site</h1>
    <nav>
      <ul>
        <li><a href="#">Início</a></li>
        <li><a href="#">Sobre</a></li>
        <li><a href="#">Contato</a></li>
      </ul>
    </nav>
  </header>

  <section>
    <article>
      <h2>Título do Artigo</h2>
      <p>Conteúdo do artigo...</p>
    </article>
  </section>

  <footer>
    <p>&copy; 2024 Meu Site. Todos os direitos reservados.</p>
  </footer>
</body>
</html>
```

Práticas Web

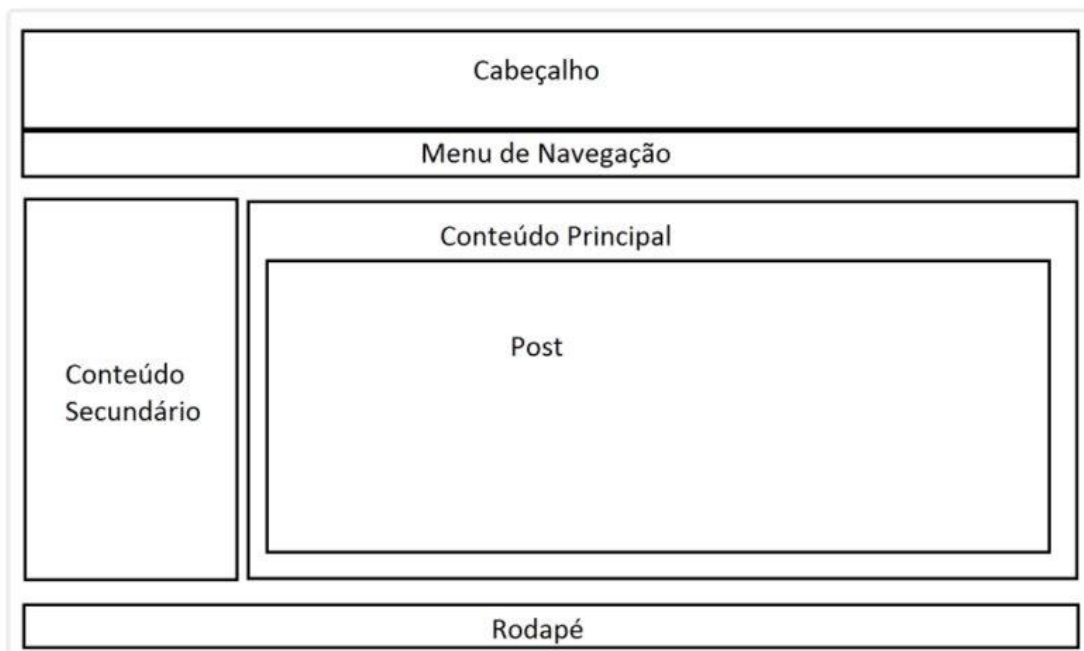


```
<div class="cabecalho" >
|   <div class="menu-navegacao" ></div>
</div>

<div class="conteudo-secundario" ></div>

<div class="conteudo-principal" >
|   <div class="post" ></div>
</div>

<div class="rodape" ></div>
```

```
<header class="cabecalho" >
|   <nav class="menu-navegacao" ></nav>
</header>

<aside class="conteudo-secundario" ></aside>

<main class="conteudo-principal" >
|   <article class="post" ></article>
</main>

<footer class="rodape" ></footer>
```

Práticas Web - Semântica

Div e Span: <div> não exibe nada, a menos que a usando CSS. As demais tags também são assim. Servem apenas para fazermos divisões.

- <div> é um elemento de bloco
- é um elemento de linha

Article

Define um artigo. Muito utilizado em Posts de blogs e sites de notícias.

Aside

Define um conteúdo ao lado do conteúdo da página. Muito utilizado em colunas ao lado da parte principal dos sites, onde normalmente possui a navegação, propagandas ou algo relacionado com o conteúdo principal.

Code

Define um pedaço de código de computador

Práticas Web - Semântica

Details e Summary

A tag <details> serve para acrescentar detalhes adicionais sobre algo, mas essa informação fica escondida, sendo exibida apenas quando o usuário clicar para ler.

A tag <summary> serve para exibir um título para o <details>, que ficará sempre visível.

Dialog

serve para criar modais (popups).

Footer

Define um rodapé para o documento ou seção.

Header

Define um cabeçalho para o documento ou seção.

Main

Define a parte principal de um documento. Por ser algo indicado como principal, o conteúdo dessa tag deve ser único e ela não deve ser filha de tags como <article>, <footer>, <header> ou <nav>.

Nav

Define um grupo de links de navegação. Essa tag é muito utilizada nos menus utilizados para se navegar entre as páginas de um site.

Section

Funciona como a Div, sendo mais específica. Define seções no documento, como capítulos, cabeçalhos, rodapés, etc.

"Cascading Style Sheets"

- Linguagem de estilos usados para descrever e apresentar elementos HTML
- Permite criar estilos gerais

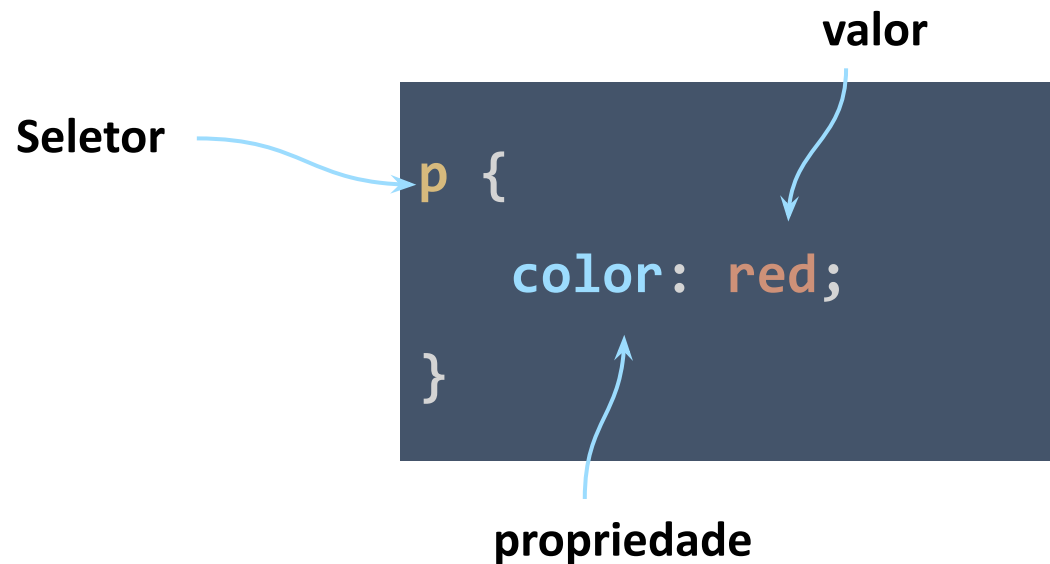
```
<h1 style="color: blue;">Titulo.</h1>
```

```
<link rel="stylesheet"  
href="style.css">
```

Adicionar CSS a um elemento HTML

- CSS Inline
- CSS interno
- CSS externo

```
<style>  
  h1 {  
    color: blue;  
  }  
</style>  
(...)  
<h1>Titulo.</h1>
```



Seletor: indica qual o elemento que queremos modificar

Propriedade: indica o que queremos alterar/editar

Valor: valor da propriedade

CSS: Cores

1. Nomes: red, blue, lightgreen, darkorange, chocolate, gold, lavender, coral, etc.

2. RGB (Red, Green, Blue)

Para se obter uma cor, passamos a quantidade de vermelho, verde e azul que queremos. O menor valor é 0 e o maior é 255.

Também há o padrão RGBA, onde A (alpha) indica a transparência da cor. O valor máximo é 1 e o menor é 0.

0: totalmente transparente; 1: totalmente opaca.

Se quisermos uma cor 50% transparente, passaríamos 0.5.

```
p {
  color: rgba(255, 153, 49, 0.5);
}
```

CSS: Cores

3. Hexadecimal: O formato hexadecimal é o mais utilizado, principalmente em softwares de edição de imagens.

Mesma ideia do RGB: passamos os valores de vermelho, verde e azul. A diferença é que as cores em hexadecimal começam com um “#”, e os números de 0 a 255 seguem o formato hexadecimal.

Branco seria #FFFFFF e preto seria #000000;

O formato hexadecimal não aceita transparência. Caso transparência seja necessária, utilize o formato RGBA.

```
p {  
    color: #FFFFFF;  
}
```

Atributos e *Selectors* CSS

```
<div class="menu">
  <!-- Menu -->
</div>
```

```
<div id="menu">
  <!-- Menu -->
</div>
```

```
<h1>
  <!-- Título -->
</h1>
```

```
.menu {
  /* Estilos */
}
```

```
#menu {
  /* Estilos */
}
```

```
h1 {
  /* Estilos */
}
```

- Para seleccionar os elementos pelo atributo **class**, utiliza-se um ponto no início: **.**
- Para seleccionar os elementos pelo atributo **id**, utiliza-se um cardinal no início: **#**
- Para seleccionar os elementos pela **tag**, coloca-se só o nome da **tag**

Atributos e *Selectors* CSS

```
<div class="menu">
  <!-- Menu -->
</div>

<nav>
  <!-- Menu -->
</nav>

<div class="menu">
  <div class="item">
    <!-- Menu -->
  </div>
</div>

<div class="menu">
  <h1>Titulo</h1>
  <p>Texto</p>
</div>
```

```
.menu, nav {
  /* Estilo aplicado aos elementos com
  classe .menu e aos elementos nav */
}

.menu > .item {
  /* Estilo aplicado aos elementos com
  classe item diretamente dentro do
  .menu */
}

.menu * {
  /* Estilo aplicado a qualquer elemento
  dentro do .menu */
}
```

Pseudo-classes

```
<div class="button">
  <!-- Botão -->
</div>

<div class="menu">
  <p class="item">1</p>
  <p class="item">2</p>
  <p class="item">3</p>
</div>
```

```
.button:hover {
  /* Estilos para o elemento com a classe
  button ao passar o cursor por cima */
}

.button:active {
  /* Estilos para o elemento com a classe
  button ao clicar em cima do elemento */
}

.item:first-child {
  /* Estilo aplicado ao elemento com a
  classe item, que é o primeiro elemento
  dentro do seu superior hierarquico */
}
```

Prioridade dos selectores

A ordem/cascata com que os seletores são utilizados é importante, mas existem outros fatores, nomeadamente (do mais prioritário ao menos prioritário):

- Se o atributo de estilo contém a keyword **!important**
- Tipo dos seletores: **id** tem precedência à **class**, **class** têm precedência às **tags**
- Ordem com que os seletores são utilizados

```
.caixa .titulo {
  color: green;
}

.titulo {
  color: blue;
}
```

```
<div class="caixa">
  <h1 id="titulo" class="titulo">Viagem</h1>
</div>
```

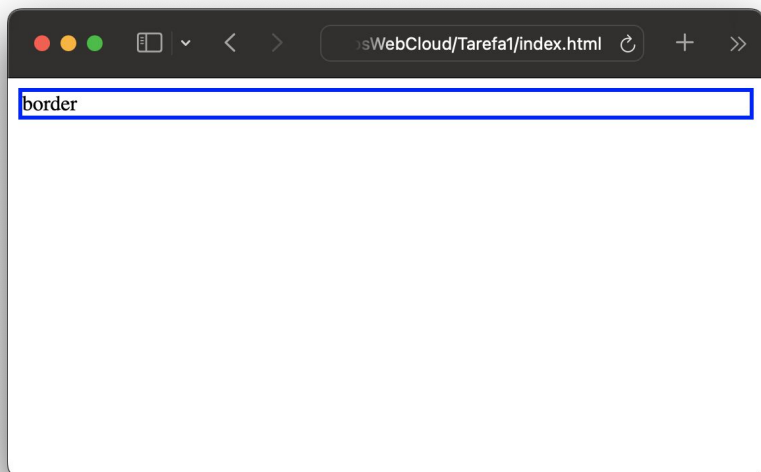


- **Color:** alterar a cor da fonte
- **Background:** mudar o fundo dos elementos
background-color:
Background-image:
- **Opacity:** influencia a visibilidade do elemento (0 - 1)

```
p {  
  color: green;  
}  
  
.titulo {  
  background-color: #00FF00;  
}
```



- **Borders:** adicionar bordas aos elementos
border-width
border-style
border-color



```
div {  
  border-width: 3px;  
  border-style: solid;  
  border-color: blue;  
}
```

none|hidden|dotted|dashed|solid|double

CSS: unidades

- **Unidades de medida:** altura, largura, tamanho de margens, posicionamento, tamanho de imagens, bordas, etc.
- As unidades são sempre colocadas após um número:

15px, 25cm, 50%

As unidades podem ser **Absolutas** ou **Relativas**.

Absolutas:

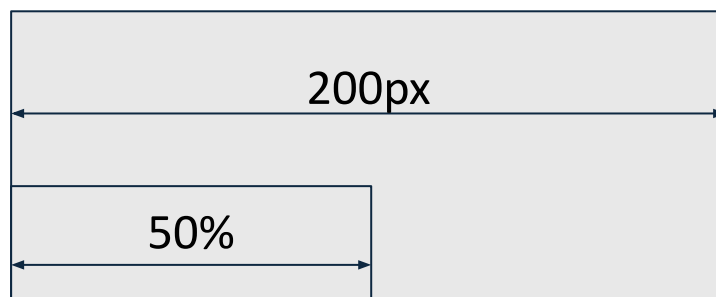
Medidas fixas. São apresentadas exatamente da mesma maneira em todos os lugares. O mais usado é o “px” (pixels):

cm - centímetros; **mm** - milímetros; **in** - polegadas; **px** - pixels

CSS: unidades

Relativas:

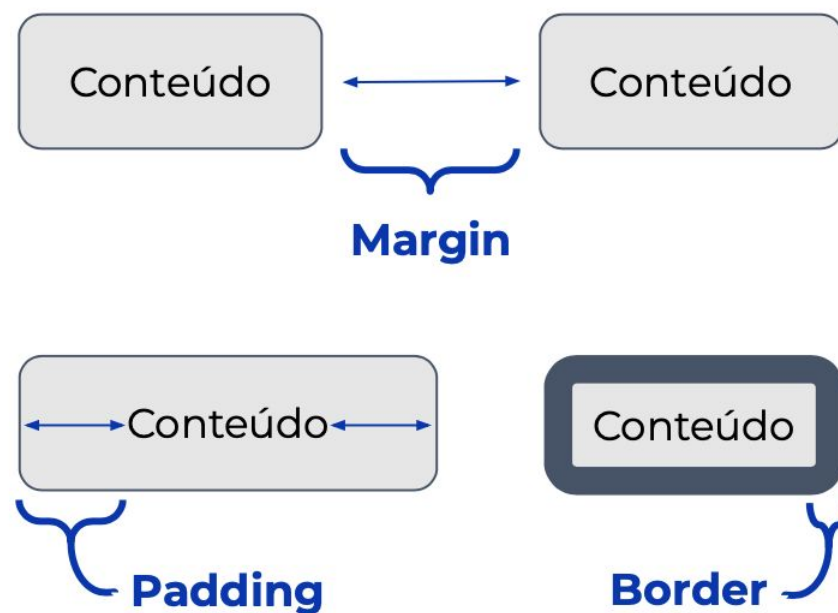
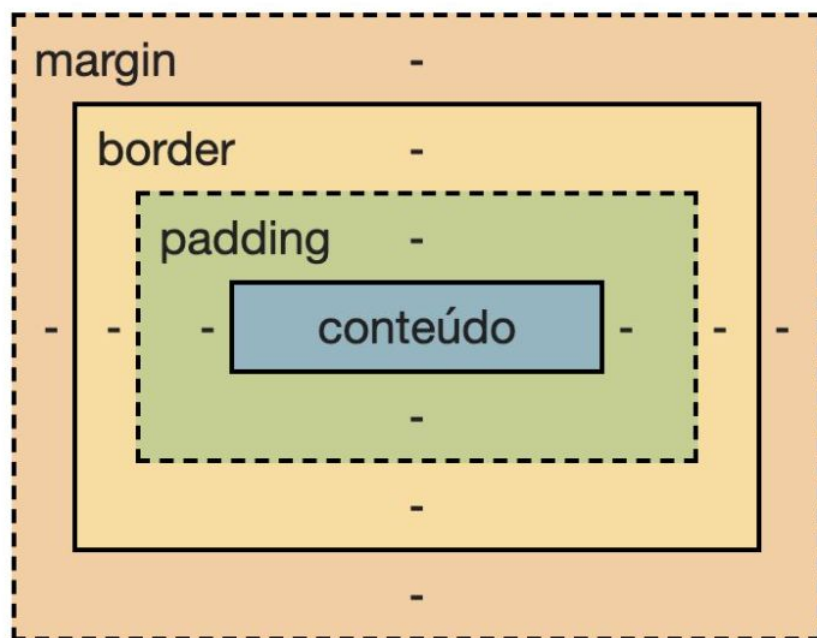
Aplica-se o valor em relação ao valor de uma outra propriedade ou elemento.



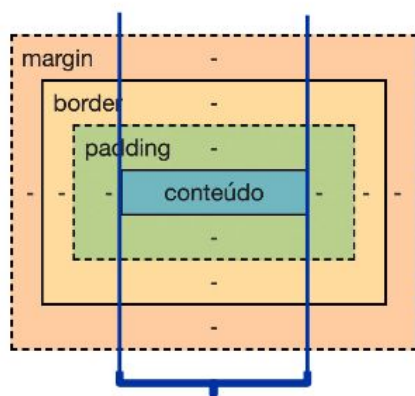
em - relativo ao tamanho da fonte do elemento. Se o tamanho da fonte for 16px, então 0.5em é igual a 8px;

rem - igual ao em, mas em relação ao tamanho da fonte do elemento raiz;

Espaçamento dos elementos

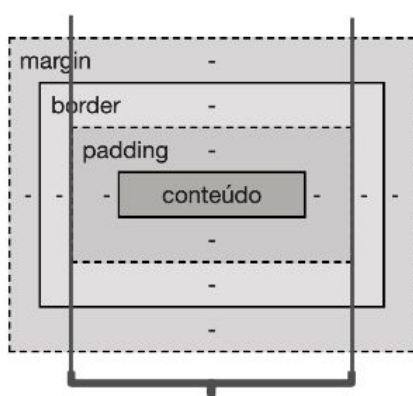


Espaçamento (box-sizing)

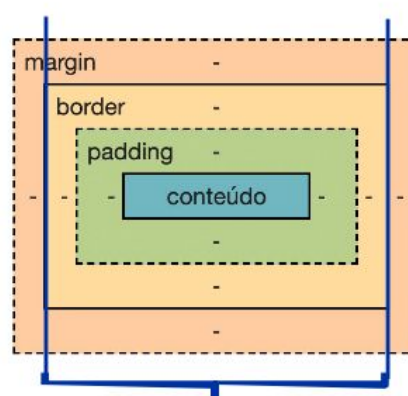


100px?
content-box

Opção por defeito



100px?
padding-box



100px?
border-box

Opção que
podemos definir /
geralmente mais
usada

Propriedade `box-sizing` permite ter em conta o *padding* e *border* quando definimos o tamanho *width* do elemento.

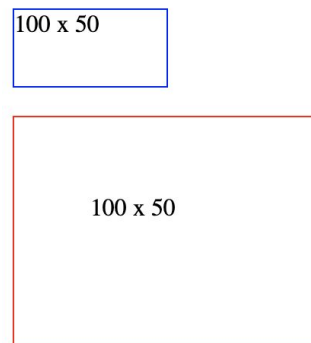
```
box-sizing:  
content-box | padding-box | border-box;
```

Tamanho elementos

- Por definição o tamanho dos elementos é calculado por:

width/height + padding + border

- Quando definimos o *width* e *height* de um elemento ele pode parecer maior do que está definido na estrutura CSS



```
<div class="elemento1">
  100x50
</div>
```

```
<div class="elemento2">
  100x50
</div>
```

```
.elemento1 {
  width: 100px;
  height: 50px;
  border: 1px solid blue;
}
```

```
.elemento2 {
  width: 100px;
  height: 50px;
  padding: 50px;
  border: 1px solid blue;
}
```

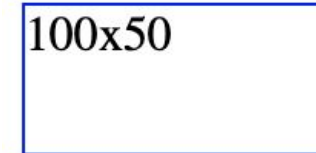
Espaçamento (box-sizing) - exemplo

```
<div class="elemento1">
  100x50
</div>
```

```
<div class="elemento2">
  100x50
</div>
```

```
.elemento1 {
  width: 100px;
  height: 50px;
  box-sizing: border-box;
  border: 1px solid blue;
}
```

```
.elemento2 {
  width: 100px;
  height: 50px;
  padding: 10px;
  box-sizing: border-box;
  border: 1px solid blue;
}
```



Tarefa 3

- Aplicar estilos CSS aos links criados na Tarefa 2 e tentar replicar o seguinte Layout/Style



CSS: atributos de texto

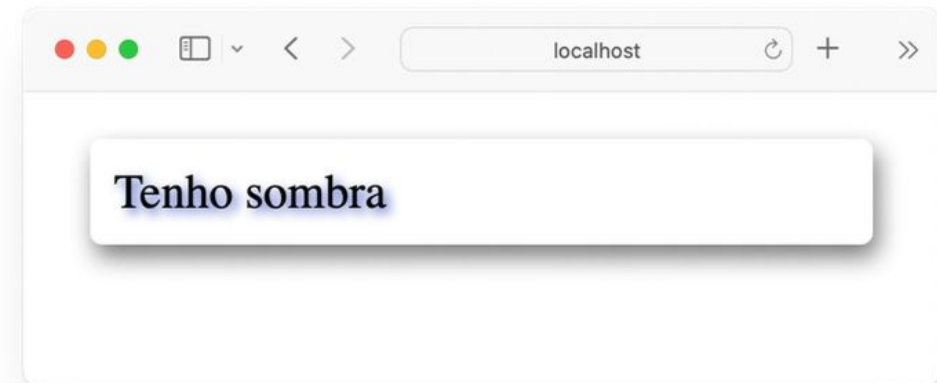
- font-size - tamanho do texto (unidades);
- text-align - left | center | right | justify;
- text-decoration - overline | line-through | underline | underline overline;
- text-transform - uppercase | lowercase | capitalize;
- letter-spacing - espaço entre letras/caracteres (unidades)
- line-height - espaço entre linhas (ex: 0.7)
- word-spacing - espaço entre palavras (unidades)

```
p {
  font-size: 20px;
  text-align: center;
  text-decoration: underline;
  text-decoration: uppercase;
  word-spacing: 10px;
}
```

This is a paragraph.

CSS: atributo box-shadow / text-shadow

- Permite adicionar uma **sombra** personalizada:
 - **text-shadow**: h-shadow v-shadow blur color;
 - **box-shadow**: h-offset v-offset blur spread color;
- Só funciona em **caixas**, mas colabora com o **border-radius**;
- Para usar em texto, é necessário o **text-shadow**.

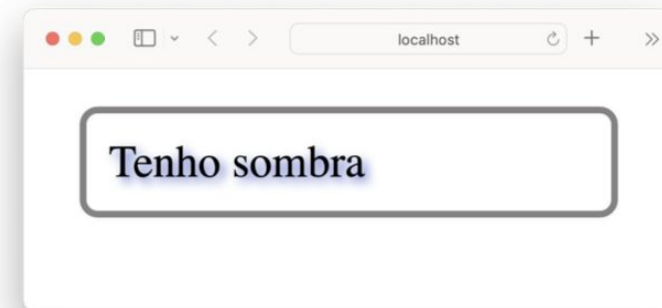


CSS: atributo box-shadow

- **box-shadow:** h-offset v-offset blur spread color;
 desvio desvio desfoque alcance cor
 vertical horizontal

- É possível jogar com o valor do blur e do spread para simular uma borda, sem afetar a dimensão do elemento.

```
box-shadow: 0 0 0 3px #797777;
```



CSS: atributo box-shadow

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: coral;  
  box-shadow: 10px 10px;  
}
```

Text color black.

```
div {  
  width: 300px;  
  height: 100px;  
  background-color: coral;  
  box-shadow: 10px 10px 20px 2px lightblue;  
}
```


CSS: atributo text-shadow

```
h1 {  
  text-shadow: 2px 2px #FF0000;  
}
```

Texto com shadow

CSS: atributo transform

- Permite **escalonar** (scale), **rodar** (rotate) ou **posicionar** (translate) livremente qualquer elemento visualmente.
- Baseia-se sempre na posição “natural do elemento” e nunca interfere com os restantes

```
<div class="transform-test">
  <h1>Bom dia!</h1>
</div>
```

```
.transform-test {
  width: 200px;
  height: 50px;
  border: 1px solid black;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Bom dia!

CSS: atributo transform

```
.transform-test h1 {  
  transform: translate(50px, 25px);  
}
```

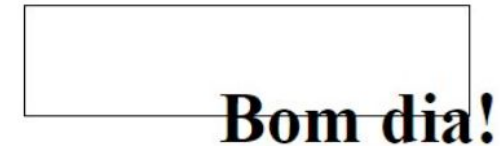
- Move o elemento da sua posição inicial (X-axis, Y-axis)

```
.transform-test h1 {  
  transform: scale(2);  
}
```

- Aumenta ou diminui o tamanho do elemento.

```
.transform-test h1 {  
  transform: rotate(45deg);  
}
```

- Roda o elemento *clockwise* ou *counter-clockwise*.



Bom dia!



Bom dia!



Bom dia!

CSS: atributo transition

- Permite alterar valores de atributos CSS de forma de forma animada
- Recebe 4 valores possíveis como argumentos:
 - **transition-property**: atributo a animar, por exemplo: **background-color**
 - **transition-duration**: duração total da animação (**3s, 0.5s, 200ms**)
 - **transition-timing-function**: tipo de animação, mais rápida ou mais lenta início ou no fim, ou uma variação qualquer, por exemplo: **ease-in-out**
 - **transition-delay**: intervalo de tempo inicial antes de começar a animação

CSS: atributo transition

```
div {  
  width: 200px;  
  height: 200px;  
  background: #000;  
  transition-property: width;  
  transition-duration: 2s;  
  transition-timing-function: linear;  
  transition-delay: 1s;  
}  
div:hover {  
  width: 300px;  
}
```

```
div {  
  transition: width 2s linear 1s;  
}
```

CSS: Fonts

O atributo `font-family` permite-nos alterar o tipo de letra a apresentar num elemento. A sintaxe permite-nos definir quais são os tipos de letra a utilizar sempre que um tipo não esteja disponível.

```
p.serif {  
  font-family: "Times New Roman", Times, serif;  
}  
p.sans_serif {  
  font-family: "Arial", Helvetica, sans-serif;  
}
```



- Também é possível utilizar tipos de letra que não estão instalados no sistema operativo, através da regra `@font-face`. Só é necessário os ficheiros de tipo de letra e a correta configuração como a apresentada em exemplo.
- Serviços como o Google Fonts permitem-nos adicionar tipos de letra facilmente aos documentos: <https://fonts.google.com>

```
@font-face {  
  font-family: NomeDaFonteAUsar;  
  src: url(letra.woff);  
}
```

CSS: Fonts

- Passos:
 - Escolher tipo de letra desejado, por exemplo:
<https://fonts.google.com/specimen/Josefin+Sans>;
 - Selecionar as variações que o site vai utilizar (bold, itálico, etc);
 - Seguir as instruções na página e incorporar o tipo de letra no documento (@import ou <link>).

```
@import
url('https://fonts.googleapis.com/css2?family=Josefin+Sans:wght@400;700&display=swap');

p {
  font-family: 'Josefin Sans', sans-serif;
}
```


CSS: Fonts

```
font-style: normal | italic | oblique;  
font-weight: normal | bold | bolder | 100 | 200 | ... (100 - 900)  
font-size: px | rem | em | xx-small ...  
font-variant: normal | small-caps | inherit
```

CSS: atributo overflow

- Quando existe a necessidade de esconder itens que saiam fora da nossa caixa de conteúdo, ou quando queremos adicionar *scroll* a uma caixa - utilizamos o **overflow**
- O overflow permite operar nos dois eixos, com o **overflow-x** e **overflow-y**

overflow: visible

Lorem ipsum dolor sit amet,
consectetur adipiscing elit, sed
do eiusmod tempor incididunt
ut labore et dolore magna
aliqua. Ut enim ad minim
veniam, quis nostrud
exercitation ullamco laboris

overflow: hidden

do eiusmod tempor incididunt
ut labore et dolore magna
aliqua. Ut enim ad minim

overflow-y: scroll

sed do eiusmod tempor
incidunt ut labore et dolore
magna aliqua. Ut enim ad

CSS: atributo cursor

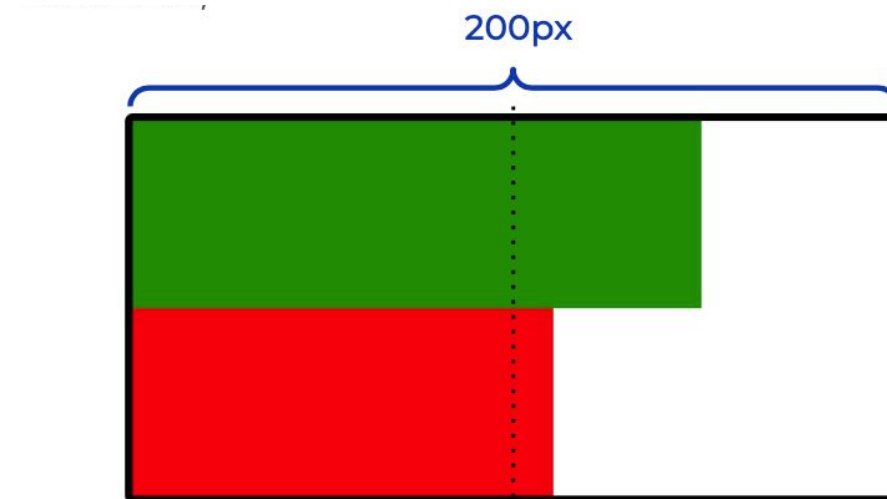
- Alterar o estilo do cursor que é apresentado quando o utilizador passa por cima do elemento
 - `cursor: pointer;` por exemplo, vai fazer com que o elemento pareça clicável.

 auto	 move	 no-drop	 col-resize
 all-scroll	 pointer	 not-allowed	 row-resize
 crosshair	 progress	 e-resize	 ne-resize
 default	 text	 n-resize	 nw-resize
 help	 vertical-text	 s-resize	 se-resize
 inherit	 wait	 w-resize	 sw-resize

CSS: calc()

- Utilizado em qualquer atributo com **valor numérico**;
- Permite efectuar **cálculos relativos**, para criar, por exemplo, tamanhos percentuais com um desvio absoluto.

```
.calc-1 {  
  background-color: green;  
  width: calc(100% - 50px);  
}  
.calc-2 {  
  background-color: red;  
  width: calc(50% + 10px);  
}
```



CSS: background-image

- É possível colocar uma imagem como fundo, com o **background-image**, criar gradientes, padrões, etc;
- Os atributos mais comuns relacionados são:
 - **background-image**
 - **background-size**
 - **background-position**
 - **background-repeat**

CSS: background-size: cover;

```
.bg-img {  
  background-image: url("img_bg.png");  
  background-size: cover;  
  background-position: center;  
}
```



CSS: background-size: contain;

```
.bg-img {  
  background-color: #6c6c6c;  
  background-image: url("img_bg.png");  
  background-size: contain;  
  background-position: center;  
  background-repeat: no-repeat;  
}
```



CSS: background-position

```
.bg-img {  
  background-color: #6c6c6c;  
  background-image: url("img_bg.png");  
  background-size: contain;  
  background-position: left top;  
  background-repeat: no-repeat;  
}
```



Centrar elementos

- `text-align: center;` permite centrar elementos *inline* contidos no nosso elemento

```
<div class="menu">  
    
  <h1>Titulo</h1>  
  <p>Fotografias</p>  
  <p class="videos">Videos</p>  
</div>
```

```
.menu {  
  width: 500px;  
  border: 1px solid black;  
  margin-left: 20px;  
  border-radius: 5px;  
  text-align: center;  
}  
.videos {  
  width: 200px;  
}
```



Centrar elementos

- Para centrar um elemento com `display: block` como um elemento `<div>` podemos utilizar `margin: auto;`
- Só é possível usar esta propriedade se o elemento for mais pequeno do que o elemento *'parent'*

```
.videos {  
  width: 200px;  
  margin: auto;  
}
```

iscte INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Titulo

Fotografias

Videos

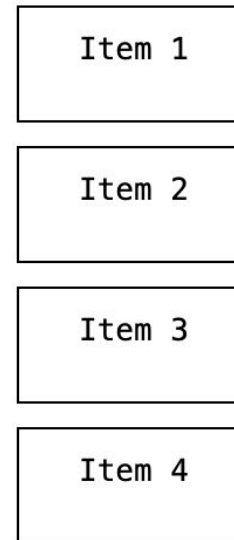
CSS atributo display

- Atributo usado para controlar o layout e o comportamento de um elemento HTML.
- Especifica como o elemento deve ser exibido no documento
- Valores mais comuns:
 - `display: none;`
 - `display: inline;`
 - `display: inline-block;`
 - `display: block;`
 - `display: flex`

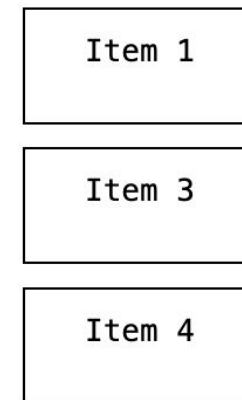
CSS atributo display: none

- Elemento não é exibido na página;
- Elemento é removido por completo e não ocupa espaço na página.

```
<div class="container display-example">
  <div class="item item1">Item 1</div>
  <div class="item item2">Item 2</div>
  <div class="item item3">Item 3</div>
  <div class="item item4">Item 4</div>
</div>
```



```
.item2 {
  display: none;
}
```

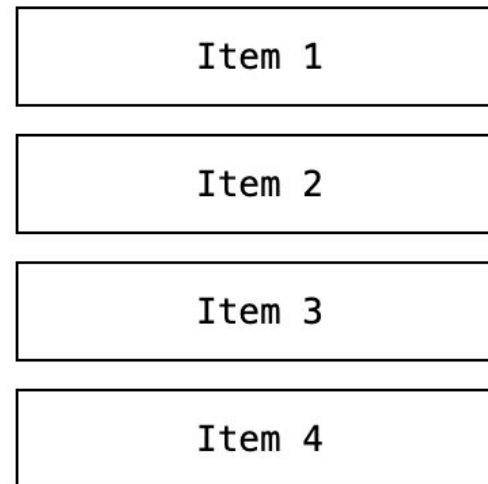


CSS atributo `display: block`

- Elemento começa sempre numa nova linha;
- Largura ocupa todo o espaço disponível, ou seja, largura é igual à do seu *'parent'* elemento.

```
.container {
  width: 200px;
}

.item {
  display: block;
  border: solid 1px black;
  margin: 10px;
  padding: 10px;
  text-align: center;
}
```

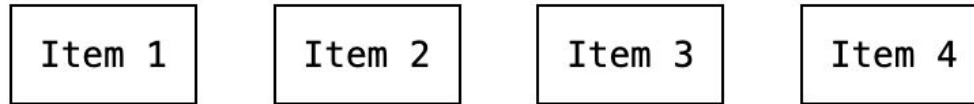


Elementos em *block*:

`<div>` ; `<h>` ; `<p>` ;
`<section>`

CSS atributo display: inline

- Elemento é exibido *inline*, sem haver quebra de linha (new line)
- A dimensão do conteúdo é calculada a partir do tamanho do conteúdo (atributos width e height são ignorados)



Elementos *inline*:

 ; <a> ;

```
.item {
  border: ► solid 1px ■ black;
  margin: ► 10px;
  padding: ► 10px;
  text-align: center;
  display: inline;
  width: 100px; ⓘ
  height: 50px; ⓘ
}
```

CSS atributo display: inline-block

- Combina os comportamentos de *inline* e *block*;
- O elemento não tem de começar no início de uma linha, mas a dimensão pode ser alterada

Item 1

Item 2

Item 3

Item 4

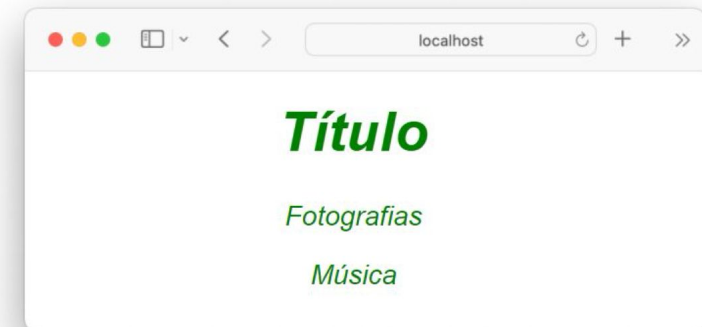
```
.item {
  border: ► solid 1px ■ black;
  margin: ► 10px;
  padding: ► 10px;
  text-align: center;
  display: inline-block;
  width: 100px;
  height: 50px;
}
```

CSS atributos de texto

Atributos de estilo de texto, por norma propagam para os elementos *child*

```
<div class="menu">
  <h1>Titulo</h1>
  <p>Fotografias</p>
  <p>Música</p>
</div>
```

```
.menu {
  color: green;
  font-family: Arial;
  text-align: center;
  font-style: italic;
}
```



CSS atributos de layout

Por norma os atributos de formatação, layout, não propagam para os elementos *child*

```
<div class="menu">
  <div class="logo">
    
  </div>
  <h1>Titulo</h1>
  <p>Fotografias</p>
</div>
```

```
.menu {
  border: 1px solid black;
  margin-left: 20px;
  border-radius: 5px;
  height: 200px;
}
```



CSS cores

As cores podem ser aplicadas a elementos HTML para definir a sua aparência visual, como:
fundo (background); borda (border); sombra (shadow); etc.

Hexadecimal:

```
.div {  
  color: #FF0000;  
  background-color: #00FF00;  
}
```

RGBA:

```
.div {  
  color: rgba(255, 0, 0, 0.5);  
}
```

Nome:

```
.div {  
  color: red;  
  background-color: green;  
}
```

RGB:

```
.div {  
  color: rgb(255, 0, 0);  
  background-color: rgb(0, 255, 0);  
}
```

Propriedade `position`: pode ser utilizado para modificar a posição de um elemento, podendo afetar a forma como este interage com o restante layout.

Existem 5 valores para o posicionamento:

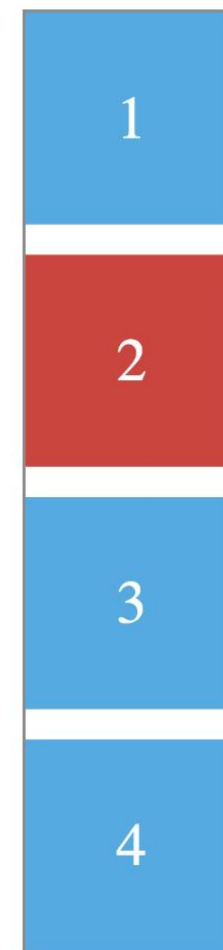
- `static` (default)
- `relative`
- `fixed`
- `absolute`
- `sticky`

CSS Position - static

- Posição inicial de cada elemento, ignora atributos como o top, bottom, left ou right;
- Um elemento com `position: static;` não está posicionado de maneira especial. Fica apenas posicionado de acordo com o fluxo normal da página.

```
.box {  
  position: relative;  
  display: inline-flex;  
  border: #8B8B8B solid 1px;  
  flex-direction: column;  
}
```

```
.static {  
  position: static;  
  background-color: #db3636;  
  top: 15px;  
  left: 15px;  
}
```

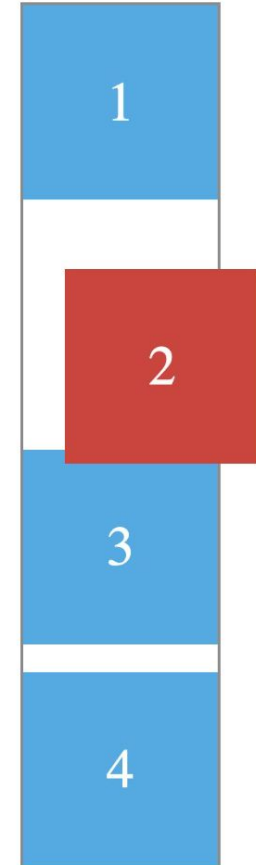


CSS Position - relative

- O elemento é posicionado relativamente à sua própria posição;
- Os atributos `top`, `bottom`, `left` e `right`, terão efeito na posição final do elemento, mas os outros elementos não serão afetados;

```
.box {
  position: relative;
  display: inline-flex;
  border: #8B8B8B solid 1px;
  flex-direction: column;
}
```

```
.static {
  position: static;
  background-color: #db3636;
  top: 15px;
  left: 15px;
}
```



CSS Position - *fixed*

- O elemento é posicionado relativamente à janela;
- Os elementos são removidos do fluxo da página, pelo que o espaço que ocupavam é libertado;

```
.fixed {
  position: fixed;
  background-color: #db3636;
  top: 15px;
  right: 15px;
}
```

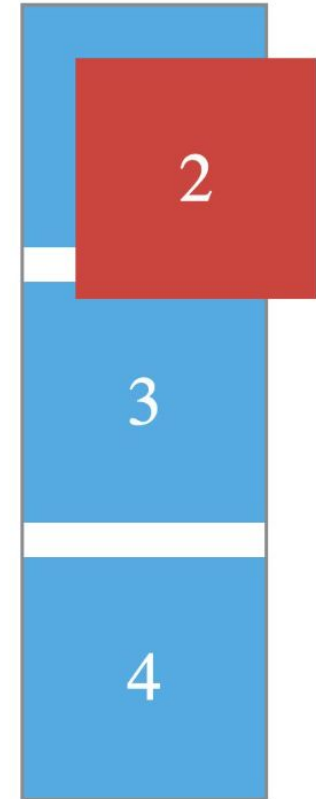


CSS Position - absolute

- O elemento é posicionado relativamente ao seu “parente”
- Os elementos são removidos do fluxo da página, por isso os elementos podem ficar sobrepostos (*não ocupam espaço no layout*)

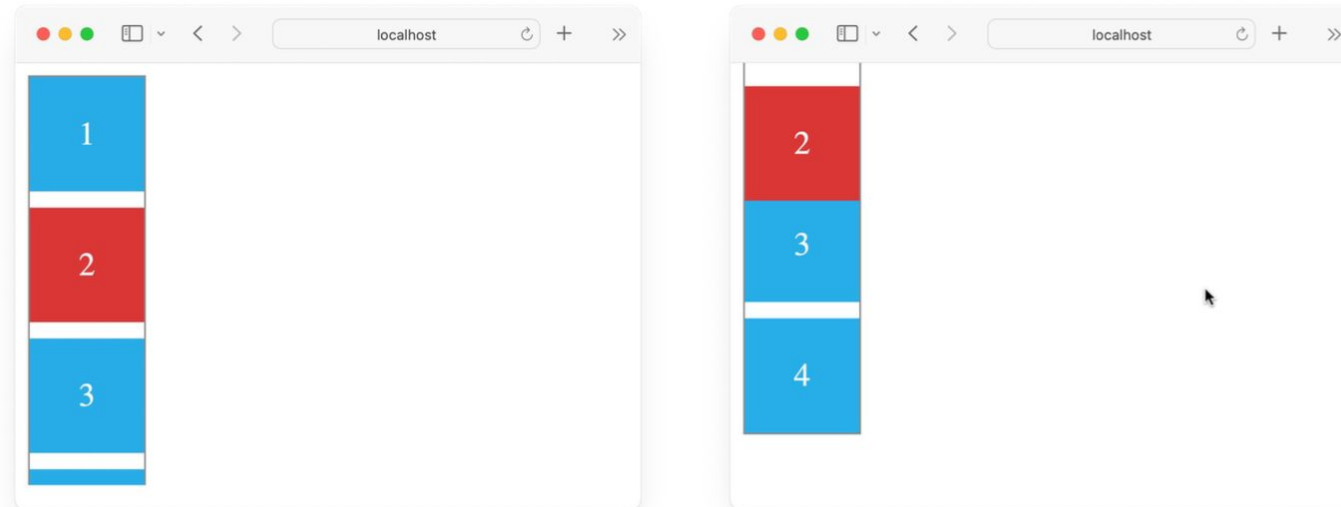
```
.box {
  position: relative;
  display: inline-flex;
  border: #8B8B8B solid 1px;
  flex-direction: column;
}
```

```
.absolute {
  position: absolute;
  background-color: #db3636;
  top: 15px;
  left: 15px;
}
```



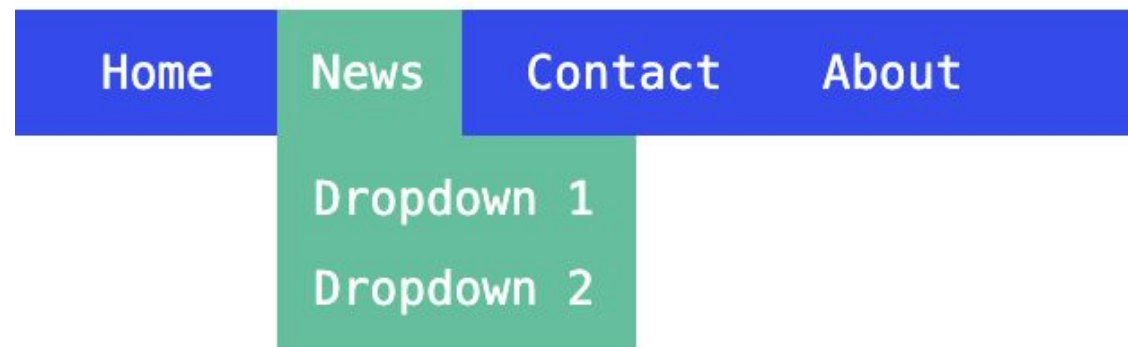
CSS Position - sticky

- O elemento tem comportamento de posição relativa até chegar ao limite da janela ao fazermos scroll. Depois muda para um comportamento de posição fixa.



Exercício 1

- Criar uma *navigation bar* com elemento HTML *unordered list* ()
 - *Unordered list* composta por items () e hyperlinks (<a>);
 - Cor Azul: #444bf6
 - Cor Verde: #5bcaa7

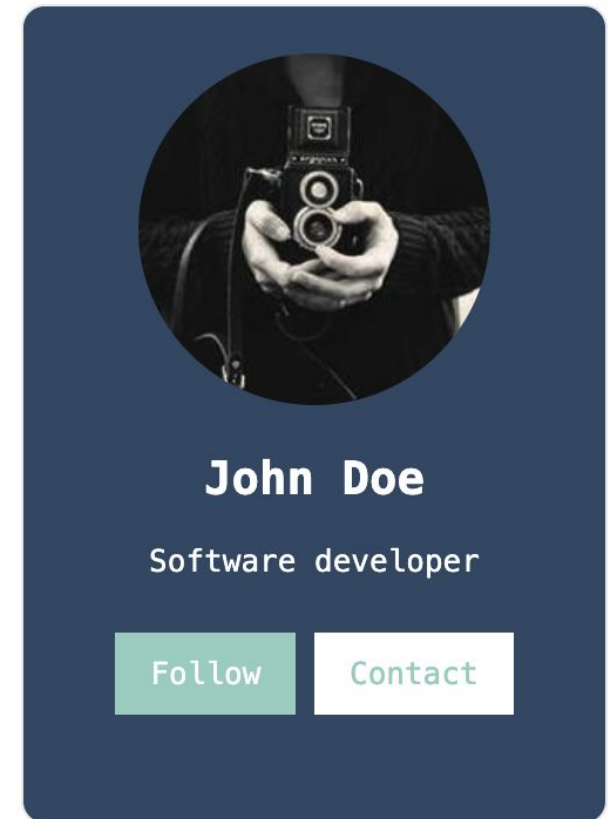
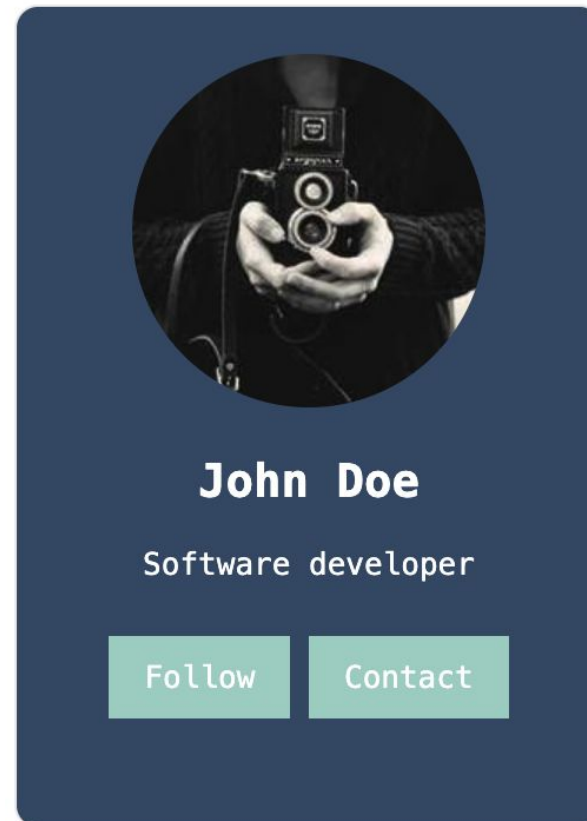


Exercício 2

- Criar um *User Profile Card*:
 - Background: #385170
 - Hyperlinks: #9fd3c7

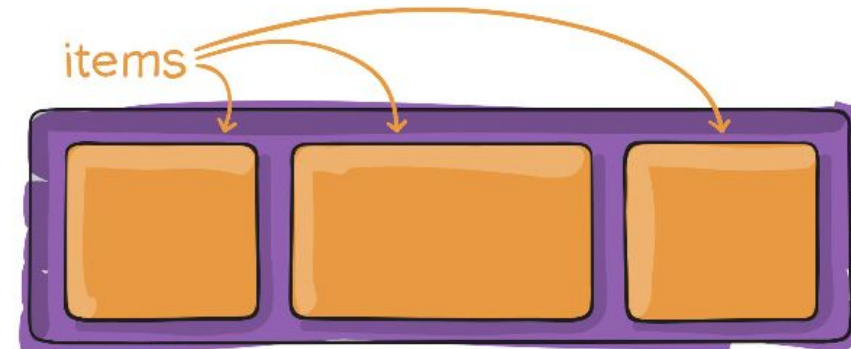
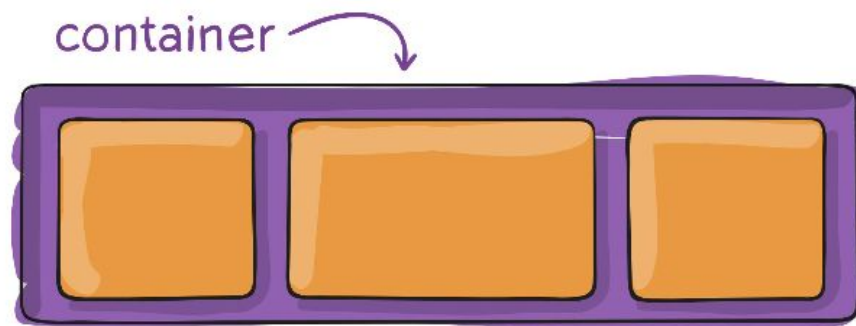
Dica: para a imagem usar um “container” e aplicar propriedade `background-image`

https://www.w3schools.com/cssref/pr_background-image.php



- O Flexbox é o modelo moderno para criação de layouts;
- Permite criar layouts de forma eficiente, alinhando e distribuindo os elementos dentro de uma caixa/container, mesmo quando a sua dimensão é desconhecida e dinâmica.
- O seu propósito principal é facilitar o posicionamento dos elementos de forma a preencher o espaço disponível da melhor forma possível.

- O Flexbox inclui todo um conjunto de propriedades, sendo que algumas são afetas à caixa exterior (flex container) e outras são direcionadas aos elementos internos (flex items)
- Para utilizar este modelo precisamos de definir o atributo **display: flex** no container;



Flexbox

Flex container pode ter as seguintes propriedades:

```
.container {  
  display: flex;  
}
```

- flex-direction
- flex-wrap
- flex-flow
- justify-content
- align-items
- align-content



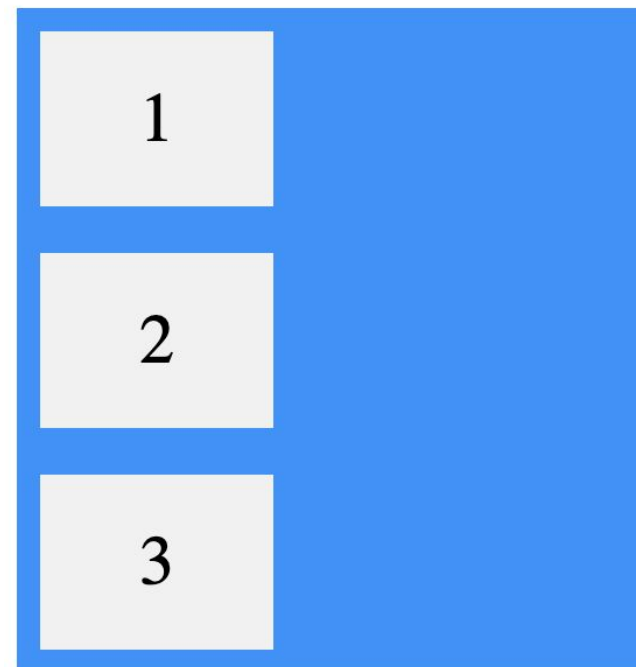
Flexbox

`flex-direction: row | column | row-reverse | column-reverse`

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```



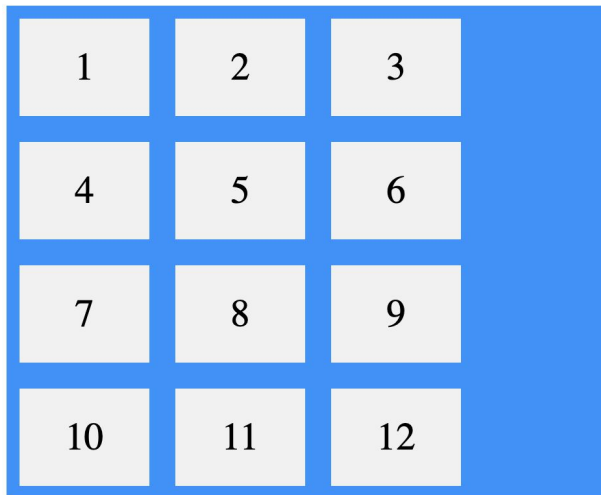
```
.container {  
  display: flex;  
  flex-direction: column;  
}
```



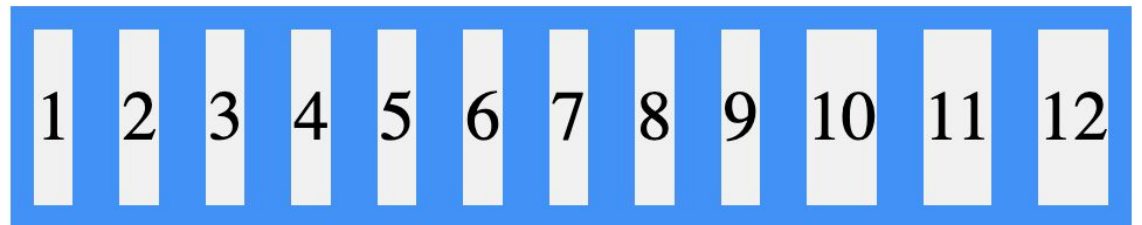
Flexbox

`flex-wrap: wrap | nowrap (default)`

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```



```
.container {  
  display: flex;  
  flex-wrap: nowrap;  
}
```



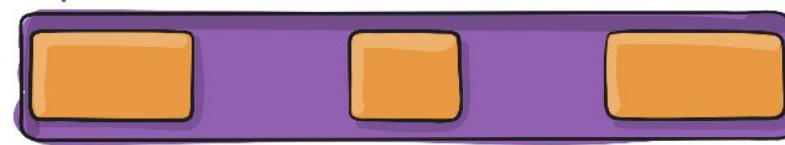
Flexbox

justify-content

flex-start



space-between



flex-end



space-around



center



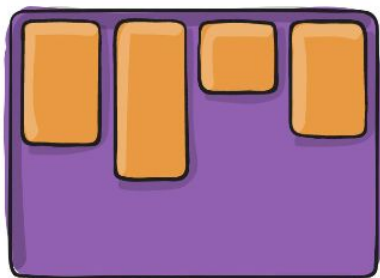
space-evenly



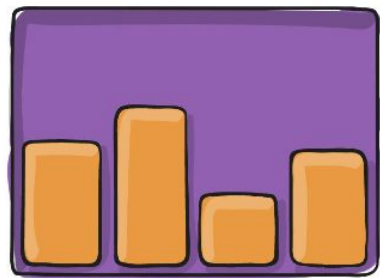
Flexbox

align-items

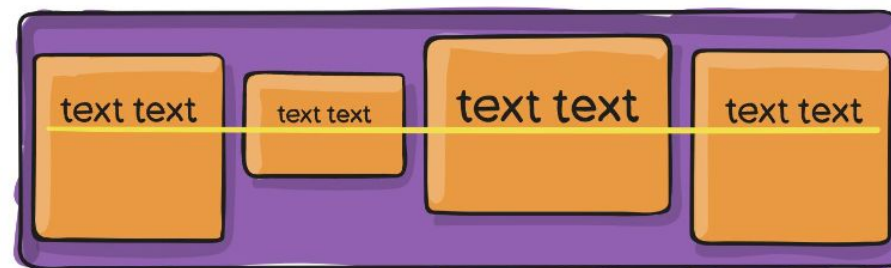
flex-start



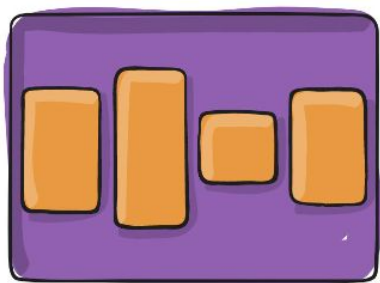
flex-end



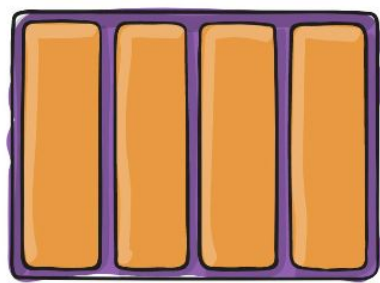
baseline



center



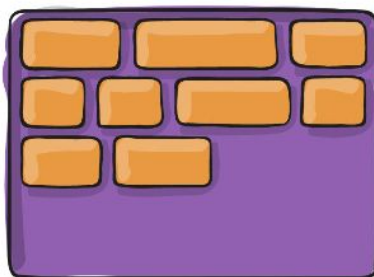
stretch



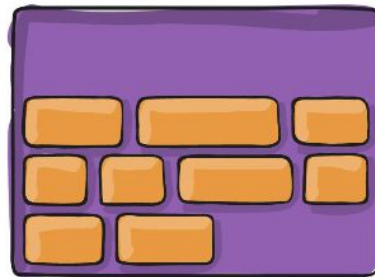
Flexbox

align-content (apenas quando é utilizado o flex-wrap)

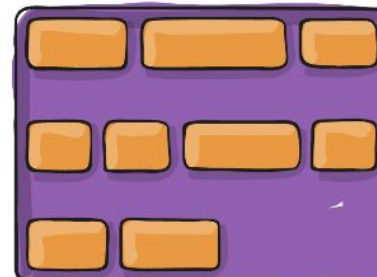
flex-start



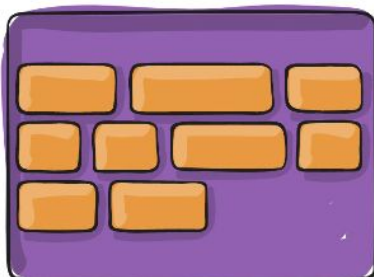
flex-end



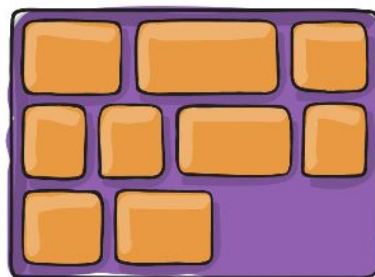
space-between



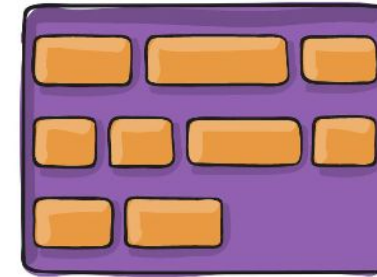
center



stretch



space-around



Flexbox - items

- Os elementos diretos de um container *flex*, são automaticamente classificados em *flex items*
- Flex items podem ter as seguintes propriedades;
 - order
 - flex-grow
 - flex-shrink
 - flex-basis
 - flex
 - align-self

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
</div>
```

Flexbox - items

- order

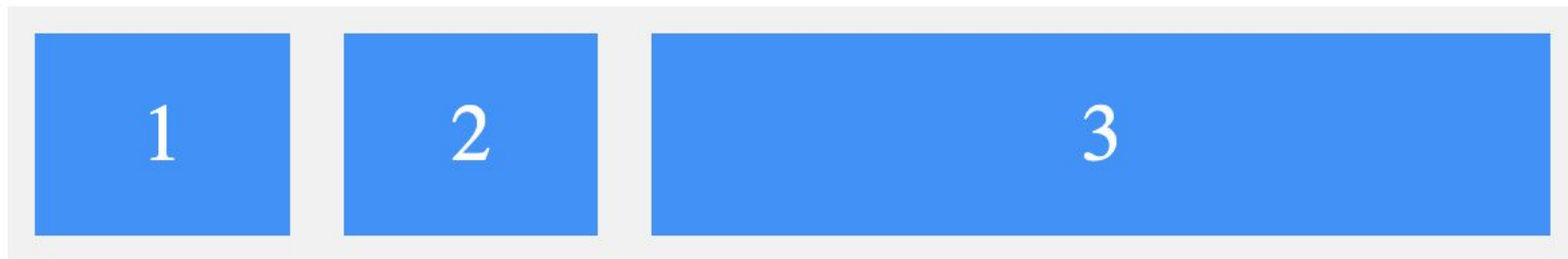
```
<div class="flex-container">  
  <div style="order: 3">1</div>  
  <div style="order: 2">2</div>  
  <div style="order: 4">3</div>  
  <div style="order: 1">4</div>  
</div>
```



Flexbox - items

- **flex-grow:** especifica quanto um elemento cresce relativo aos outros elementos; Elementos ocupam o tamanho máximo do container.

```
<div class="flex-container">
  <div style="flex-grow: 1">1</div>
  <div style="flex-grow: 1">2</div>
  <div style="flex-grow: 4">3</div>
</div>
```

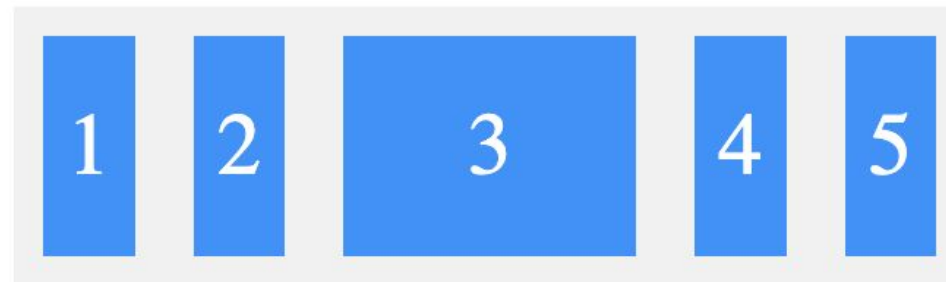


Flexbox - items

- **flex-shrink**: especifica quanto um elemento pode encolher caso não exista espaço para todos os elementos.

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-shrink: 0">3</div>
  <div>4</div>
  <div>5</div>
</div>
```

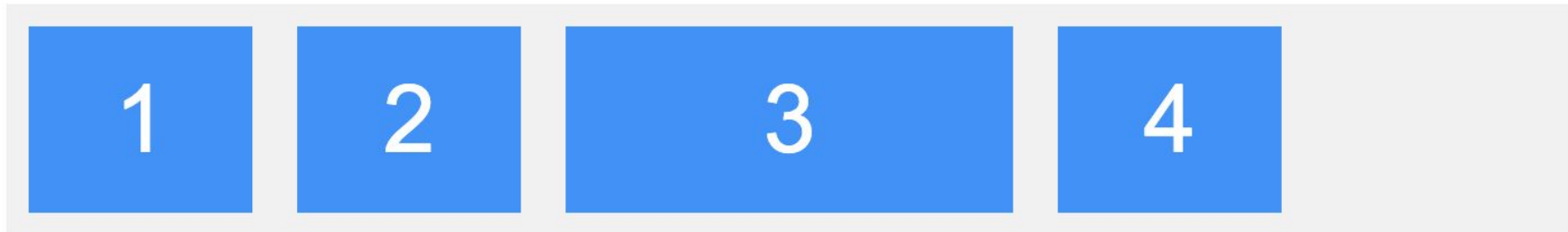
```
.flex-container div {
  width: 100px;
}
```



Flexbox - items

- **flex-basis**: especifica o tamanho inicial do elemento.

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-basis: 200px">3</div>
  <div>4</div>
</div>
```

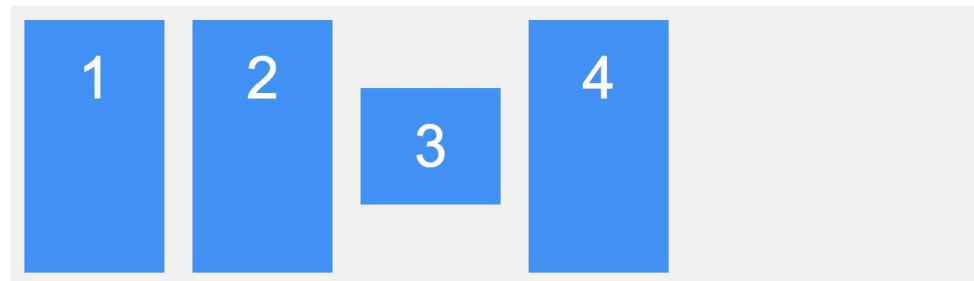


Flexbox - items

- **align-self**: especifica o alinhamento para o elemento

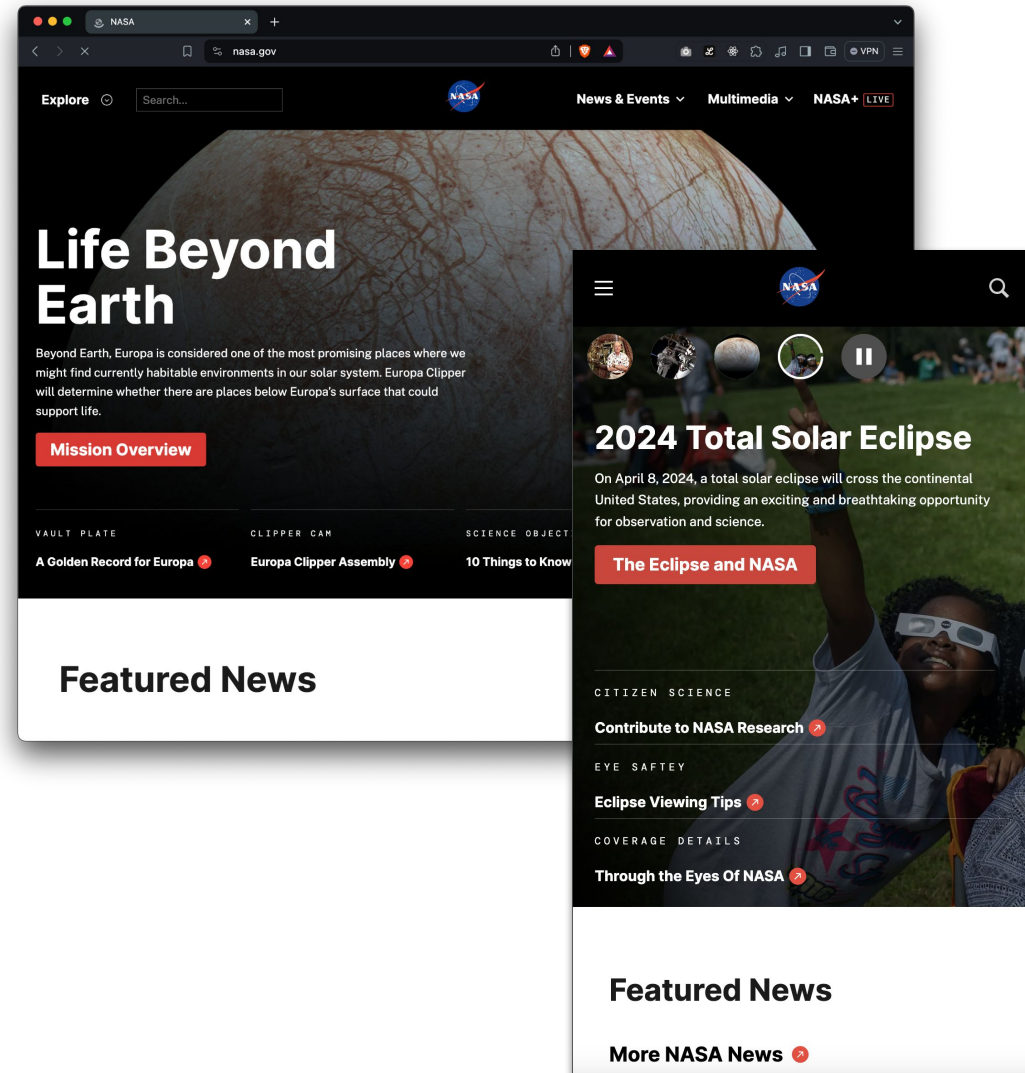
```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="align-self: center">3</div>
  <div>4</div>
</div>
```

```
.flex-container {
  display: flex;
  height: 200px;
}
```



Media Queries

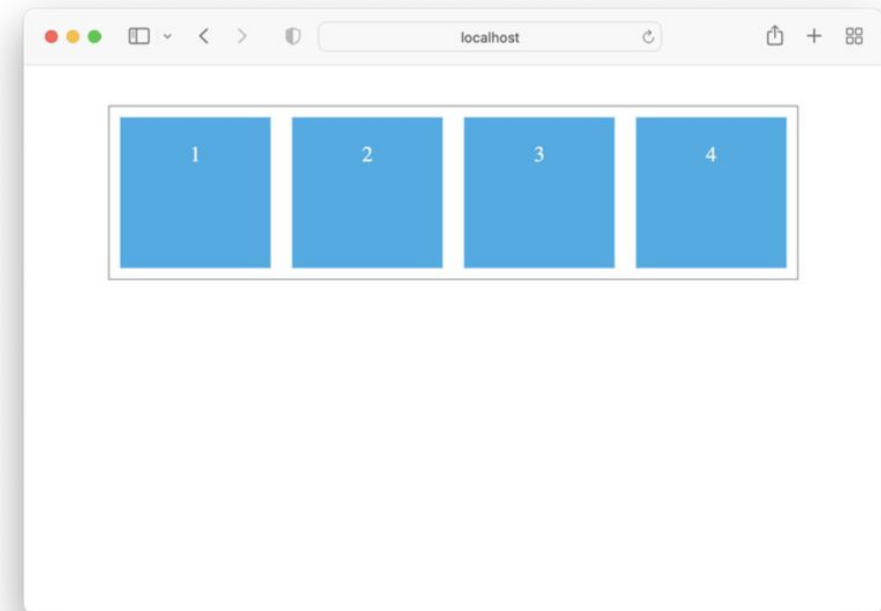
- Como podemos programar a nossa página para aparecer bem em todos os ecrãs?
- Utilizar larguras máximas não é suficiente e é necessário definir condições concretas para como o layout é apresentado em ecrãs de dimensões diferentes.



Media Queries

- É habitual que uma página apresente uma disposição horizontal de elementos em ecrãs mais largos.
- Como podemos alinhar aqueles elementos verticalmente quando o ecrã fica menos largo?

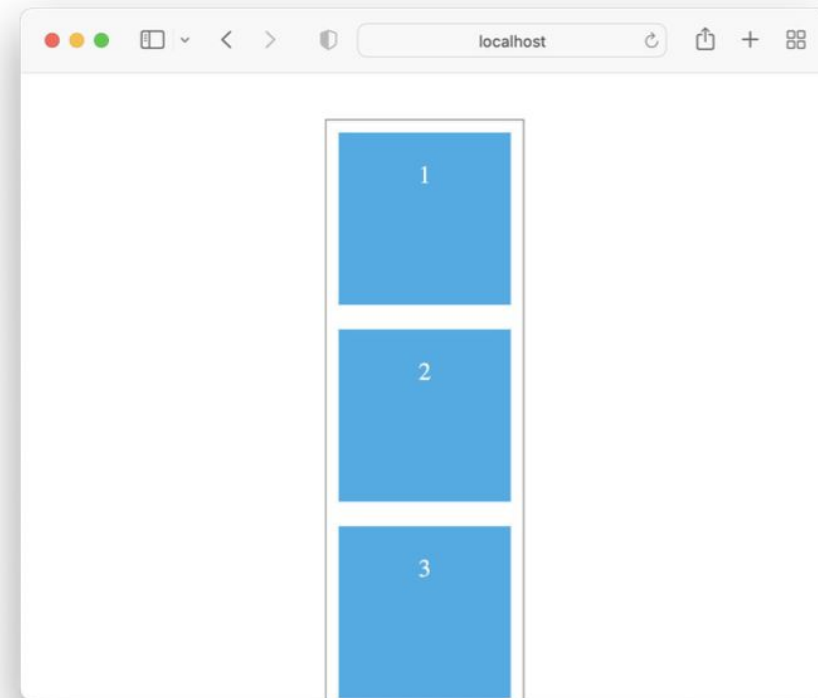
```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}
```



Media Queries

```
.container {  
  position: relative;  
  display: flex;  
  border: #8B8B8B solid 1px;  
  margin: 30px;  
  gap: 10px;  
  padding: 10px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

width <= 768px



Media Queries

- Existe um conjunto universal de valores que definem as quebras nas páginas

Dispositivos muito pequenos (600px ou menos)	@media screen and (max-width: 600px) { . . . }
Dispositivos pequenos (600px ou mais)	@media screen and (min-width: 600px) { . . . }
Dispositivos médios (768px ou mais)	@media screen and (min-width: 768px) { . . . }
Dispositivos grandes (992px ou mais)	@media screen and (min-width: 992px) { . . . }
Dispositivos muito grandes (1200px ou mais)	@media screen and (min-width: 1200px) { . . . }

Media Queries

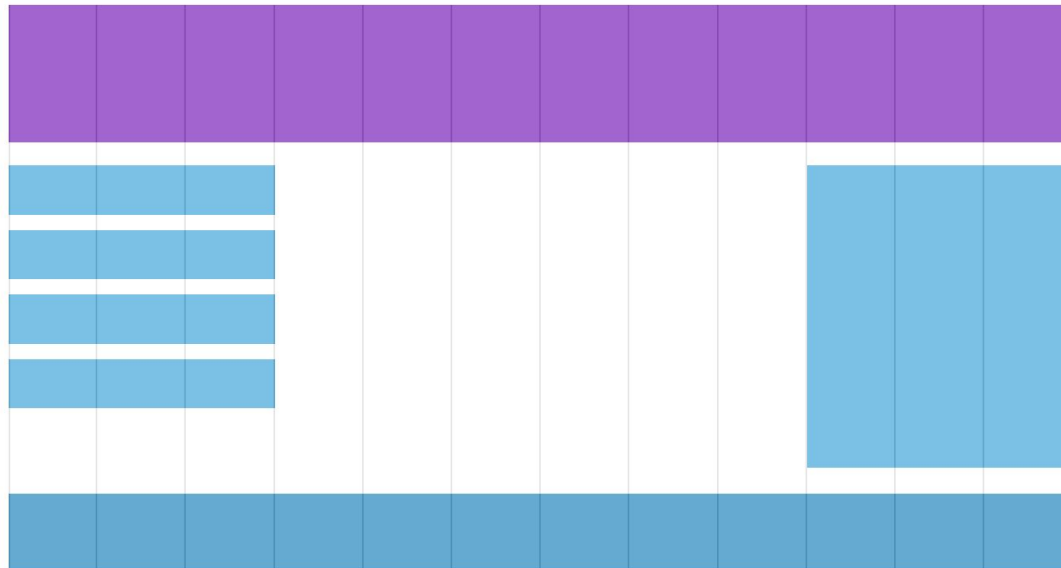
- Para a página ser corretamente apresentada nos dispositivos móveis é necessário adicionar a seguinte tag <meta> dentro do head:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- Caso esta linha não seja utilizada, o dispositivo vai considerar uma largura de janela maior e poderá não aplicar as regras de responsabilidade.

Media Queries

- Para simplificar o sistema, podemos dividir a página em colunas - **Grid-view**.



```
.col-1 {width: 8.33%;}  
.col-2 {width: 16.66%;}  
.col-3 {width: 25%;}  
.col-4 {width: 33.33%;}  
.col-5 {width: 41.66%;}  
.col-6 {width: 50%;}  
.col-7 {width: 58.33%;}  
.col-8 {width: 66.66%;}  
.col-9 {width: 75%;}  
.col-10 {width: 83.33%;}  
.col-11 {width: 91.66%;}  
.col-12 {width: 100%;}
```

Exercício

- box-shadow
- Google Font (para o título)
- background-image
- Flexbox containers
- transition: change background color of button on hover

#e97132ff

FOR SALE

#757575

Consultar



FOR SALE

Vila in Sesimbra

 Sesimbra, Portugal

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor.



Price
\$1000

Consultar

WWW

- *"World Wide Web"* (Rede Mundial de Computadores)
- Rede mundial que liga os computadores uns aos outros
- Permite a troca de informações/documentos na Internet através do Hypertext Transfer Protocol (HTTP)
- A visualização e navegação das informações/documentos é feita através do navegador - *browser*

WWW - Arquitetura

Construído em torno de 3 conceitos:

- **URL** (Uniform Resource Locator) - endereços que permitem identificar cada elemento (documentos, imagens, videos, etc.) de forma única;
- **HTTP** (Hypertext Transfer Protocol) - protocolo principal usado para transferir dados. Especifica como é feito o envio de informação entre cliente-servidor;
- **HTML** (Hypertext Markup Language): define a estrutura e os conteúdos de uma página web (texto, imagens, links, etc.)

Conteúdos

Onde estão os conteúdos?

- Podem estar em máquinas diferentes;
- Em pastas diferentes dentro da mesma máquina;
- Como podemos localizar os conteúdos/recursos?

A localização dos recursos (imagens, documentos, vídeos, áudios, etc.) é feita através do **URL**:

`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

protocolo

máquina/host

caminho/path

documento

DNS

`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

máquina/host

Para encontrar a "máquina" onde está o ficheiro (imagem1.jpeg) primeiro temos de saber o IP da máquina;
O DNS (Domain Name System) é a solução - O DNS traduz nomes de domínio em endereços IP;

Exemplo:

- google.pt: 142.250.201.67
- maquina_dominio.pt: 193.136.189.102

Endereço IP - DNS

IP (Internet Protocol) - número de identificação de cada máquina na internet

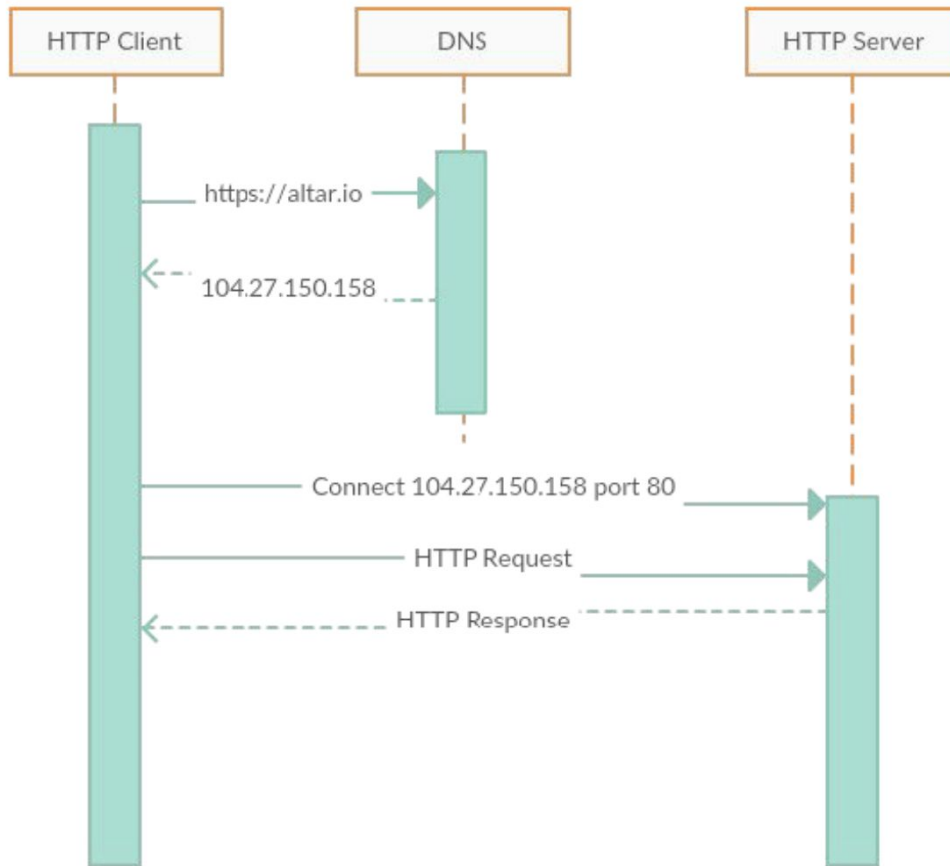


DNS foi criado para facilitar o acesso aos diferentes servidores/máquinas da rede, sem termos de saber o IP.

DNS - catálogo de endereços IP que localiza os servidores com base no nome de domínio;



Encontrar o conteúdo

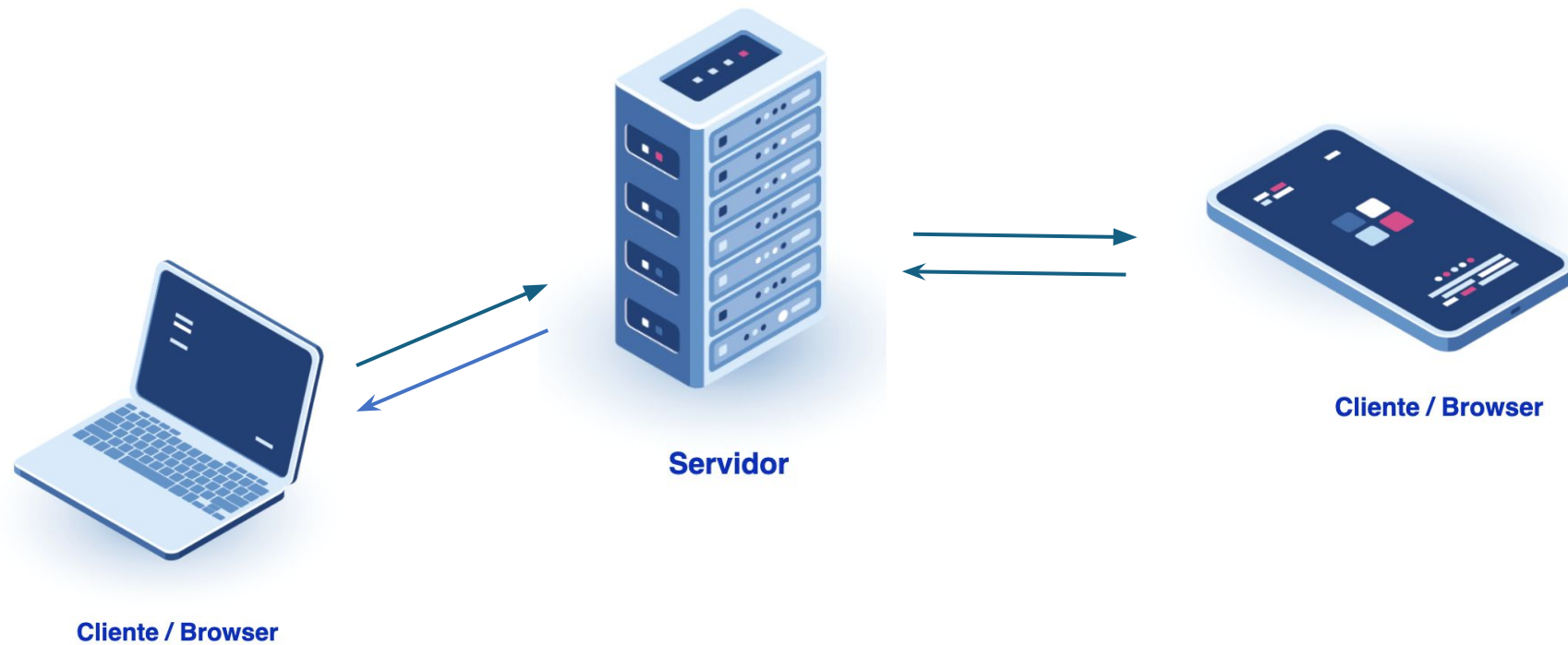


- O navegador da web envia uma solicitação DNS para um servidor DNS.
- O servidor DNS consulta sua base de dados ou outros servidores DNS para encontrar o endereço IP associado ao nome de domínio fornecido.
- Uma vez que o endereço IP é localizado, o servidor DNS retorna essa informação para o navegador.
- O navegador pode então estabelecer uma conexão com o servidor web utilizando o endereço IP retornado e solicitar os recursos desejados.

`http://maquina_dominio.pt/imagens/2024/imagem1.jpeg`

`http://193.136.189.102/imagens/2024/imagem1.jpeg`

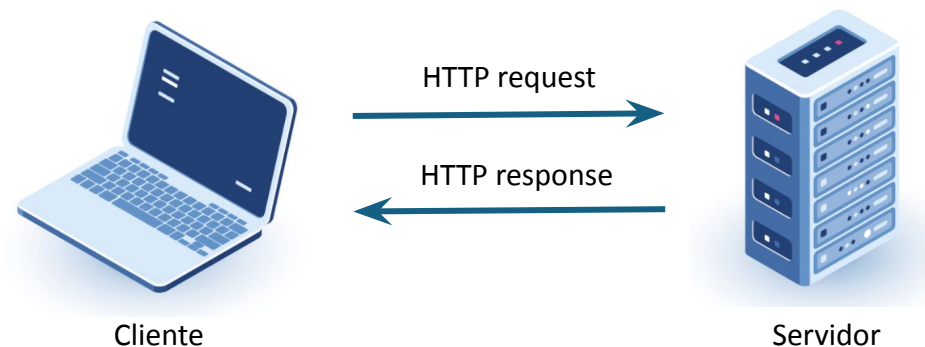
Arquitetura Cliente - Servidor



HTTP

"Hypertext Transfer Protocol"

- Implementado para permitir a comunicação *web browser / web server*
- Transmissão de documentos *hypermedia*
- Modelo cliente-servidor: cliente faz um *request* e espera pela resposta do servidor
- *"stateless protocol"* - não mantém/guarda o estado entre duas conexões
- Define como clientes Web requisitam páginas aos servidores e como estes transferem a informação

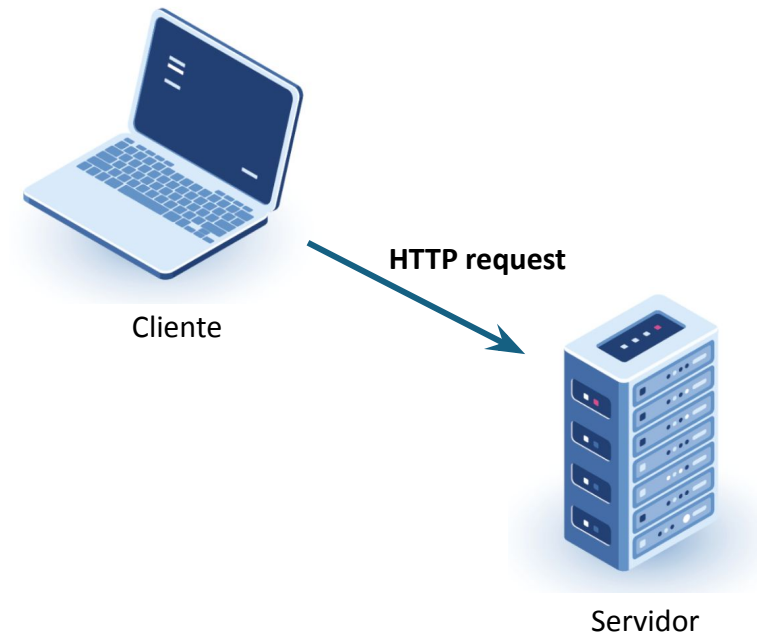


HTTP – Request

HTTP Request – mensagem que um cliente envia a um servidor para requisitar recursos: páginas web, imagens, documentos, etc.

HTTP Request é composto por:

- Tipo de pedido: tipo de pedido efetuado ao servidor (GET, POST, etc.)
- URL: Endereço
- Cabeçalhos: informações adicionais a enviar ao servidor (Cache, Language, Content-type, etc.)
- Corpo: Dados enviados ao servidor para tratamento



HTTP – Tipos de pedido

GET	Obter/pedir um recurso/dados ao servidor
POST	Criar um recurso/dados no servidor
PUT	Update um recurso existente no servidor
DELETE	Delete dados no servidor
PATCH	Update um recurso existente - não cria esse recurso
CONNECT	Efetuar conexão entre servidor (TCP/IP Tunnel)
OPTIONS	Pedir informações sobre comunicações a um <i>endpoint</i>
TRACE	Usado para teste/diagnóstico durante desenvolvimento
HEAD	Obter mensagem de cabeçalho sem os recursos

HTTP – GET

GET	Obter/pedir um recurso/dados ao servidor
-----	--

Método GET é usado para solicitar dados a um recurso específico no servidor. Cliente usa este método quando quer receber dados, como páginas web, imagens, conteúdos, etc.

- Pode ser cacheado pelo browser;
- Aparece no histórico de navegação
- Têm limite de dimensão
- Usado apenas para obter dados, não modificar

HTTP – GET

GET	Obter/pedir um recurso/dados ao servidor
-----	--

Parâmetros GET:

- Parâmetros são enviados no endereço URL para especificar com maior detalhe os recursos a serem obtidos:

`http://host.com/books?ano=2000&author=JohnDoe`

Estrutura dos parâmetros GET
key/value pair

HTTP – POST

POST

Criar um recurso/dados no servidor

Método POST é usado para criar ou alterar um recurso que está no servidor.

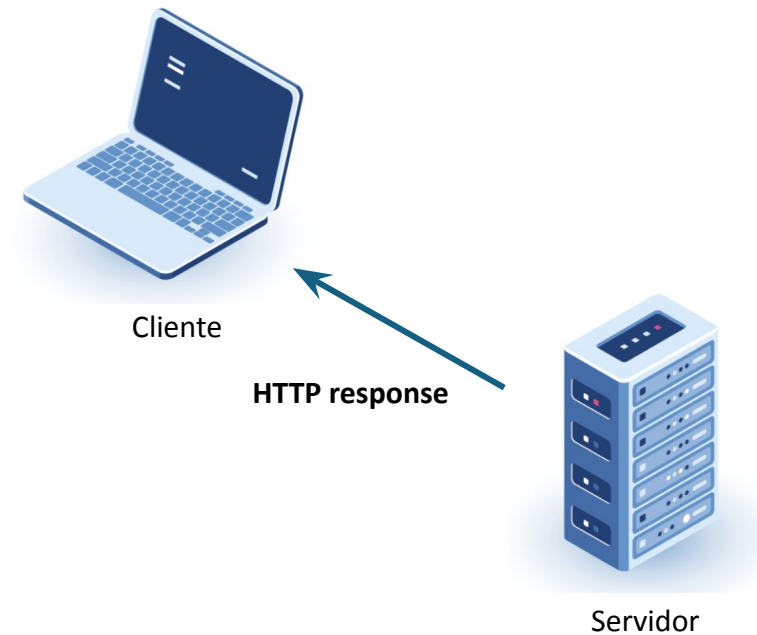
- Não podem ficar na cache do browser
- Não têm limite de dimensão (mas servidor pode limitar)

HTTP – Response

HTTP Response – mensagem que o servidor envia ao cliente depois de receber e processar o pedido

HTTP Response é composto por:

- Resultado/Status: código HTTP representativo de sucesso ou de erros na processamento do pedido
- Cabeçalhos: informações adicionais a enviar ao cliente para entender a resposta (codificação de conteúdo, etc.)
- Corpo: conteúdo da resposta HTTP



HTTP – Status

Código de 3 dígitos que indica o resultado do pedido HTTP.

Exemplo de códigos comuns:

1XX Information	102 Processing
2XX Success	200 Success / OK
3XX Redirection	301 Permanent Redirect
4XX Client Error	404 Not Found 403 Forbidden
5XX Server Error	500 Internal Server Error 504 Gateway timeout

HTTP – Cabeçalhos

Exemplo de cabeçalho de um pedido GET:

```
GET /home.html HTTP/1.1
Host: developer.mozilla.org
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:50.0) Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://developer.mozilla.org/testpage.html
Connection: keep-alive
Upgrade-Insecure-Requests: 1
If-Modified-Since: Mon, 18 Jul 2016 02:36:04 GMT
If-None-Match: "c561c68d0ba92bbeb8b0fff2a9199f722e3a621a"
Cache-Control: max-age=0
```

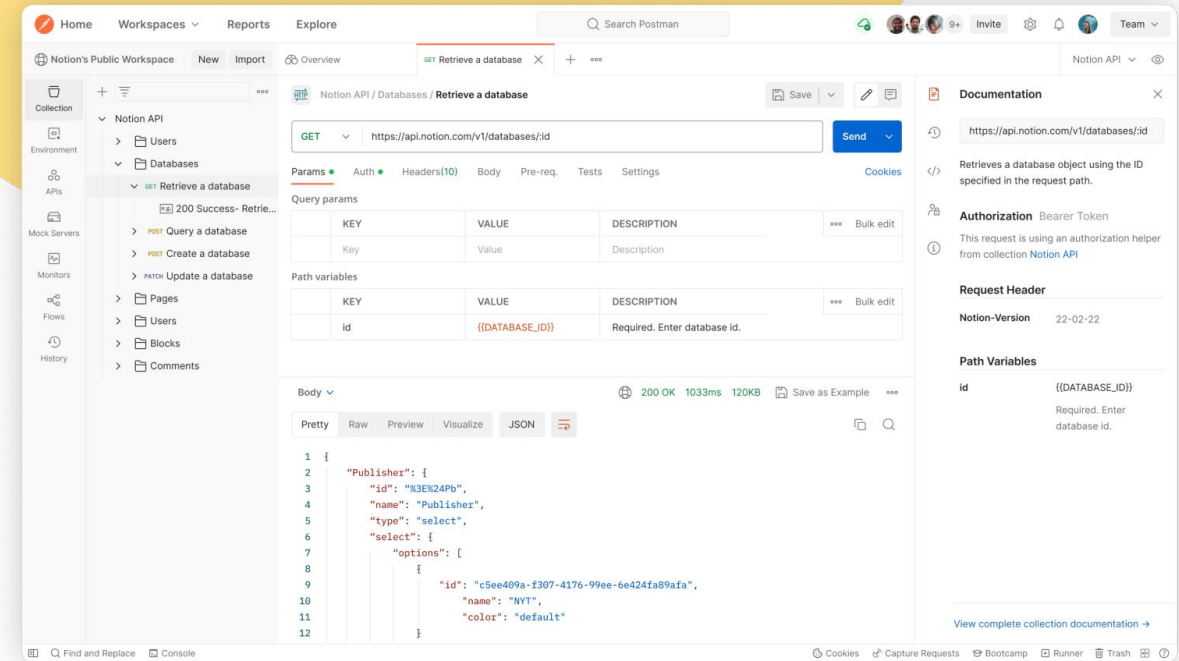
Exemplo de cabeçalho de resposta:

```
200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Mon, 18 Jul 2016 16:06:00 GMT
Etag: "c561c68d0ba92bbeb8b0f612a9199f722e3a621a"
Keep-Alive: timeout=5, max=997
Last-Modified: Mon, 18 Jul 2016 02:36:04 GMT
Server: Apache
Set-Cookie: mykey=myvalue; expires=Mon, 17-Jul-2017 16:06:00 GMT; Max-Age=31449600;
Path=/; secure
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding
X-Backend-Server: developer2.webapp.scl3.mozilla.com
X-Cache-Info: not cacheable; meta data too large
X-kuma-revision: 1085259
x-frame-options: DENY
```

POSTMAN

Ferramenta de suporte para testar, desenvolver e documentar APIs (Application Programming Interfaces)

<https://www.postman.com/>



Tarefa

- Preparar pequena biblioteca de testes com o POSTMAN
- Usar API <https://rickandmortyapi.com/>
- Numa nova *Collection* criar os seguintes endpoints:
 - Listar todos os episódios
 - Listar detalhes de um episódio
 - Obter detalhes de vários personagens

<https://rickandmortyapi.com/documentation/>

HTML

"HyperText Markup Language"

- *Markup Language*, define a estrutura de como é apresentado o conteúdo/texto
- *HyperText*, define as ligações entre documentos, dentro do mesmo site ou para sites externos
- Descreve a estrutura de páginas Web
- Conjunto de diferentes elementos que dizem ao *browser* como apresentar ou dispor os conteúdos
- Não é considerado um linguagem de programação
- Os conteúdos ou a estrutura de cada página é definida por **tags**

Elementos HTML

Elemento HTML é definido por uma *tag* inicial, o conteúdo e uma *tag* final

```
<h1> O meu primeiro texto. </h1>
```

Tag de
abertura

Conteúdo

Tag de
fecho

- Exceções:
 e <hr>

Elementos HTML

Dentro de cada elemento HTML podemos ter outros elementos:

```
<p> Parágrafo sem ênfase no meio </p>
```

Output:

Parágrafo sem ênfase no meio.

```
<p> Parágrafo com <strong>ênfase</strong> no meio </p>
```

Parágrafo com **ênfase** no meio.

Template HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Título da Página</title>
  </head>
  <body>
    <!-- Conteúdo da Página -->
  </body>
</html>
```

`<html>` tag principal que encapsula todo o documento

`<head>` tag que encapsula a informação "invisível" do documento (ficheiros CSS, código Javascript, etc.)

`<body>` tag de conteúdo que encapsula todo o conteúdo que pretendemos apresentar

Atributos HTML

Atributos HTML adicionam informação extra a um elemento HTML

```
<p style="color: red"> Este texto é vermelho. </p>
```

Output:

Este texto é vermelho.

- Atributos têm de ser especificados na tag inicial
- Atributos são normalmente definidos com *"name/value" pairs*

Atributos HTML

Atributos definem comportamentos, formatação, informação adicional, etc:

Exemplos de atributos:

- *style* – permite aplicar estilos CSS ao elemento;
- *href* – define o URL de uma hiperligação;
- *target* – alterar o comportamento de um link (abrir num novo separador, etc.);
- *src* – define o URL de localização de uma image, video ou áudio;
- *class*
- *id*

Elementos HTML

Títulos HTML: `<h1>` até `<h6>`, sendo `<h1>` a tag mais importante.

```
<h1> Título. </h1>
<h2> Título. </h2>
<h3> Título. </h3>
```

Parágrafos HTML:

```
<p> Parágrafo. </p>
```

Links/Hiperligações HTML: o link de destino é especificado no atributo `href`

```
<a href="https://www.google.com"> Google </a>
```

Imagens HTML: `src` especifica o caminho até à imagem ; `alt` texto alternativo

```

```


Elemento DIV

- Permite agrupar múltiplos elementos HTML – *container*
- Usado para estruturar o layout das páginas
- Permite aplicar diferentes estilos a todos os elementos dentro do *container*

```
<div>  
  <h1>Titulo.</h1>  
  <p>Parágrafo</p>  
  <a href="link.com">Link</a>  
</div>
```

Titulo.

Parágrafo

[Link](#)

Atributo class

- Especifica o nome da *class* que pode ser usado por um ou vários elementos HTML
- Normalmente usado para especificar determinados estilos CSS
- Permite o acesso e/ou manipulação através de JavaScript

```
<div class="city">  
  <h1>Lisboa</h1>  
  <p>Capital de Portugal.</p>  
</div>
```

```
.city {  
  background-color: tomato;  
  color: white;  
  border: 2px solid black;  
  margin: 20px;  
  padding: 20px;  
  width: 100px;  
}
```



Atributo class (cont.)

- Elementos HTML podem ter mais de uma *class*
- Elemento terá os estilos definidos de todas as suas

```
<h2 class="city main">London</h2>  
<h2 class="city">Paris</h2>  
<h2 class="city">Tokyo</h2>
```

London

Paris

Tokyo

Atributo id

- Identifica um determinado elemento HTML ou uma secção da página
- Só pode existir um id único por página

```
<section id="ola">  
  <h2>Olá Mundo!</h2>  
</section>  
  
<section id="outro_id">  
  <p>Esta é outra secção!</h2>  
</section>
```

Hyperlinks

- Possibilita o encaminhamento para outra página
- Conteúdo que é "clicável" e encaminhado para outra localização

```
<a href="https://www.google.com">Google</a>
```

Atributo *href* – define o endereço do encaminhamento

Conteúdo - pode ser texto ou imagens

Pode ser encaminhado para um elemento dentro da própria página

Listas

- *Unordered list:*
- *Ordered list:*

```
<ul>
  <li>Google</li>
  <li>Microsoft</li>
  <li>IBM</li>
</ul>
```

Output:

- Google
- Microsoft
- IBM

```
<ol>
  <li>Google</li>
  <li>Microsoft</li>
  <li>IBM</li>
</ol>
```

Output:

1. Google
2. Microsoft
3. IBM

Imagens

```

```



https://www.w3schools.com/html/html_images.asp

Tarefa 1

- Criar uma página (index.html) com o seguinte conteúdo:
- Imagem clicável com link para o Google



Tarefa 2

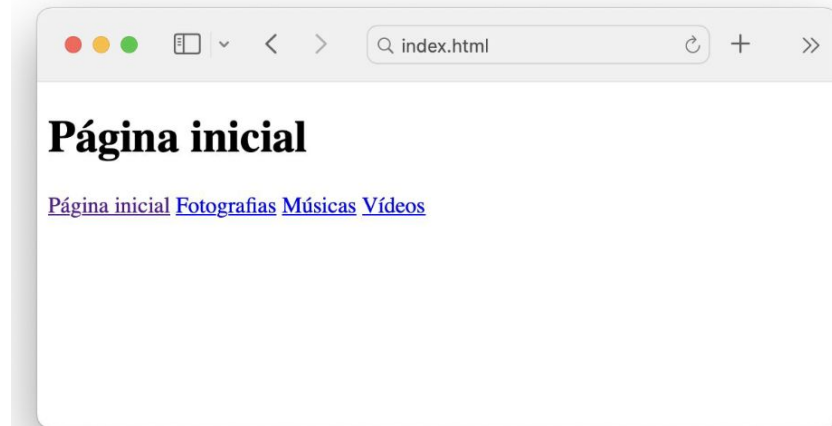
- Criar um conjunto de 4 ficheiros HTML (4 páginas)
- Criar link com navegação para as páginas

index.html - Página inicial

fotografias.html - Página Fotografias

musicas.html - Página Músicas

videos.html - Página Videos



Questões?

CSS

"Cascading Style Sheets"

- Linguagem de estilos usados para descrever e apresentar elementos HTML
- Permite criar estilos gerais

```
<h1 style="color: blue;">Titulo.</h1>
```

```
<link rel="stylesheet" href="style.css">
```

Adicionar CSS a um elemento HTML

- CSS Inline
- CSS interno
- CSS externo

```
<style>
  h1 {
    color: blue;
  }
</style>
(...)
<h1>Titulo.</h1>
```

Atributos e *Selectors* CSS

Adicionar CSS a um elemento HTML

- CSS Inline
- CSS interno
- CSS externo

```
<h1 style="color: blue;">Titulo.</h1>
```

```
<link rel="stylesheet" href="style.css">
```

```
<div class="menu">  
  h1 {  
    color: blue;  
  }  
</style>  
(...)  
<h1>Titulo.</h1>
```